Statistical Learning

Weijia Jia

2026-06-02

Table of contents

Preface

These are notes for STAT 432 2026 Summer at UIUC. If you have any questions please contact me.

References

Part I

Statistical learning Basics

1 Statistical Learning Basics

1.1 Introduction

Statistical learning studies how to use data to understand relationships and make predictions.

In supervised learning, a common abstract model is

$$Y = f(X) + \varepsilon,$$

where:

- X represents features;
- Y represents the response;
- f is the underlying relationship;
- ε represents random noise or unexplained variation.

The main objective is to estimate the unknown function f using observed data.

More broadly, modern statistical learning is fundamentally about learning patterns that generalize well to unseen data.

Depending on the objective, statistical learning problems can be viewed from several different perspectives.

Task	Main Question
Prediction	What will happen?
Inference	Which variables matter?
Causality	What happens if we intervene?

- Prediction mainly focuses on predictive performance and generalization. A common formulation is
$$\hat{f}(x) \approx \mathbb{E}[Y \mid X = x].$$
- Inference focuses on understanding relationships between variables and often relies on tools such as confidence intervals and hypothesis testing.
- Causality studies intervention effects and is often written as
$$\mathbb{E}[Y \mid \mathrm{do}(X = x)].$$

Important distinctions

1. Good training performance does not imply good generalization.
2. Good prediction does not imply valid inference.
3. Association does not imply causation.

Tip

In this course, we will primarily focus on **prediction** and **generalization** rather than formal statistical inference.

1.2 Types of Learning

Statistical learning problems can be divided into several major categories depending on the available data and learning objective.

We will mainly focus on supervised learning in this course, with some topics from unsupervised learning.

1.2.1 Supervised Learning

In supervised learning, we observe both predictors X and response Y.

The goal is to predict or explain the response, such as house price prediction, spam detection, medical diagnosis, etc..

Supervised learning is usually divided into two major tasks:

- **regression**, where the response is quantitative;
- **classification**, where the response is categorical.

Example 1.1.

- predicting salary is a regression problem;
- predicting whether a tumor is benign or malignant is a classification problem.

1.2.2 Unsupervised Learning

In unsupervised learning, we observe predictors X but no response Y.

The goal is to discover hidden structure in the data, such as clustering, dimensionality reduction, embedding learning, etc..

1.2.3 Reinforcement Learning

In reinforcement learning, an agent interacts with an environment and learns through rewards and penalties, such as robotics, game playing, recommendation systems, etc..

1.3 Types of Data

Machine learning methods are strongly connected to data structure, and different data types often require different modeling strategies.

This course mainly focuses on **tabular data**, which is the classical setting in statistics and applied machine learning.

1.3.1 Tabular Data

Tabular data is one of the most common data formats in statistical learning. It is typically represented as

$$X \in \mathbb{R}^{n \times p},$$

where:

- n is the number of observations;
- p is the number of predictors.

Example:

Age	Income	Balance	Student	Default
23	50000	1200	Yes	No
45	90000	3000	No	Yes

In tabular data:

- rows represent observations;
- columns represent variables or features.

For supervised learning, datasets usually contain:

- a feature matrix X;
- a target variable y.

For unsupervised learning, we typically observe only the feature matrix X.

Since this course mainly focuses on tabular data, we will usually treat:

$$X \in \mathbb{R}^{n \times p}, \qquad y \in \mathbb{R}^n.$$

columns correspond to variables

The key feature of tabular data is NOT merely the matrix representation. More importantly, each column corresponds to a variable with its own semantic meaning, such as age, income, blood pressure, or transaction count.

Tip

Tabular data is particularly well suited for methods such as linear models and tree-based models. Therefore, these model families will be the main focus of this course.

1.3.2 Other Types of Data

Besides tabular data, many machine learning problems involve other data formats. We briefly mention several common examples that appear in modern learning tasks.

Image Data

Image data contains spatial structure, meaning nearby pixels are highly related.
Modern image models frequently use:

- convolutional neural networks,
- vision transformers.

A central goal is to extract meaningful visual patterns such as edges, textures, shapes, and objects.

Text Data

Text data contains sequential and contextual structure. Modern NLP is largely based on transformers and large language models.

Time Series Data

Time series data is ordered over time such as stock prices, weather data and sensor measurements.

Graph Data

Graph data represents relationships between objects, such as social networks, citation networks, molecular structures.
Modern graph learning often uses graph neural networks.

1.4 Generalization and Model Complexity

1.4.1 Training Error and Test Error

Classical regression focuses heavily on checking model assumptions, such as independence of errors, constant variance, approximate normality. These assumptions are especially important for formal inference, including confidence intervals and hypothesis testing.

In statistical learning, however, the primary goal is often **predictive performance** rather than formal inference. As a result, diagnostic tools still remain useful, but they usually play a secondary role. Their main purpose is often to detect serious issues such as strong nonlinearity, influential observations, data quality problems, severe model misspecification.

The central question becomes: How well does the model predict new observations?

Definition 1.1 (Training Error and Test Error)**.**

- The training error measures how well a model fits the training data.
- The test error measures prediction error on new observations not used to fit the model.

For example, suppose we evaluate a regression model using mean squared error (MSE).

The **training MSE** measures fit on the observed training sample, while the **test MSE** measures prediction performance on new data drawn from the same population.

A model may achieve extremely small training error while still performing poorly on unseen data. This happens because the model begins fitting random noise rather than the underlying signal.

Definition 1.2 (Overfitting)**.** Overfitting occurs when a model fits the training data very closely but fails to generalize well to new data. In this case, the model captures random noise or idiosyncrasies in the training sample rather than the underlying relationship.

Model selection methods attempt to balance model complexity and prediction accuracy in order to avoid overfitting.

1.4.2 Bias–Variance Tradeoff

A central idea behind generalization is the **bias–variance tradeoff**.

Suppose

$$Y = f(X) + \varepsilon, \qquad \mathrm{Var}(\varepsilon) = \sigma^2.$$

For a fitted estimator $\hat{f}$, the test mean squared error at x_0 satisfies

$$\mathrm{E}\left[(Y_0 - \hat{f}(x_0))^2\right] = \mathrm{Bias}^2(\hat{f}(x_0)) + \mathrm{Var}(\hat{f}(x_0)) + \sigma^2.$$

The three terms represent:

- **bias**: systematic modeling error: $\mathrm{Bias}(\hat{f}(x)) = \mathrm{E}[\hat{f}(x)] - f(x)$. It measures the systematic difference between the average prediction and the true underlying function;
- **variance**: sensitivity to the training sample: $\mathrm{Var}(\hat{f}(x))$. It measures how much the fitted model changes when the training data changes. Highly flexible models often have larger variance;
- **irreducible error**: σ^2. This represents random noise or variability in the data generation process that cannot be explained or removed, even with the true model.

Roughly speaking, as model flexibility increases:

- training error usually decreases;
- test error often first decreases and then increases.

Model selection aims to balance these two effects in order to achieve good generalization performance.

> **Main Point**
>
> Many modern methods can be viewed as ways to control the bias–variance tradeoff. Regularization, cross-validation, ensemble methods, trees, boosting, and neural networks can all be understood from this perspective.

Information Criteria and Estimated Test Error

Classical regression often estimates expected test error indirectly using only the training data.
Some commonly used criteria are:

- AIC (Akaike Information Criterion),
- BIC (Bayesian Information Criterion),
- Mallows' C_p.

These criteria reward good fit while penalizing excessive model complexity.
Although they are useful for model comparison, they still rely on approximations derived from the training sample.

2 Classification Decisions and Evaluation

Unlike regression, where metrics such as Mean Squared Error (MSE) evaluate prediction error along a continuous scale, classification evaluation is fundamentally more nuanced.

A modern statistical learning classifier often produces a score or estimated probability rather than directly outputting a categorical decision. For example, a model may estimate

$$\widehat{p}(x) = \Pr(Y = 1 \mid X = x),$$

and a separate decision rule then converts this soft prediction into a hard classification. As a result, classification performance can be evaluated from several different perspectives.

Broadly speaking, classification evaluation can be divided into three related but distinct goals.

Goal	Main question	Common tools
Decision quality	Are the final classification decisions good?	Accuracy, precision, recall, F1-score
Ranking quality	Are positive observations ranked above negative observations?	ROC curve, AUC, PR curve
Probability quality	Are predicted probabilities numerically trustworthy?	Calibration curve, Brier score, log loss

These goals are related but fundamentally different. Accurate probability estimates are the richest output: they can support ranking and threshold-based decisions. However, good decisions, good rankings, and calibrated probabilities are different evaluation goals and one does not generally imply the others.

If a model accurately estimates the true conditional probability distribution $\Pr(Y \mid X)$, then it naturally induces good rankings and supports effective decision-making across many possible thresholds or cost functions. However, the converse implications generally fail. Optimizing only a threshold-based metric such as accuracy provides no guarantee that the underlying probabilities are calibrated or that the ranking behavior is reliable.

In practice, however, decision quality is often easier to achieve and is frequently the primary objective in applied statistical learning. Many real-world applications ultimately require

concrete actions, such as approving a loan, detecting fraud, or diagnosing disease. Consequently, threshold-based metrics such as accuracy, precision, recall, and F1-score often play the central role in applied machine learning.

> **Note**
>
> Classification evaluation can become quite sophisticated and may involve ranking performance, probability calibration, cost-sensitive learning, threshold optimization, and decision theory.
> In this course, however, our primary goal is to understand the core ideas of statistical learning and predictive modeling rather than to develop a complete theory of statistical decision making. Therefore, we will mainly focus on accuracy and closely related threshold-based classification metrics.

2.1 Decision quality

Decision quality evaluates the final hard classifications after applying a threshold to predicted scores or probabilities.

This perspective focuses on whether the model makes useful decisions in practice. Metrics such as accuracy, precision, recall, and F1-score belong to this category.

2.1.1 Binary Classification Setup

Suppose the response is binary:

$$Y \in \{0, 1\}.$$

We call $Y = 1$ the positive class and $Y = 0$ the negative class.

A classifier produces a predicted label

$$\hat{Y} \in \{0, 1\}.$$

The four possible outcomes are summarized by the confusion matrix.

	Predicted positive	Predicted negative
True positive class	TP	FN
True negative class	FP	TN

Here, TP means true positive, FP means false positive, FN means false negative, TN means true negative.

The confusion matrix is the starting point for most threshold-based classification metrics.

2.1.2 Accuracy

Accuracy is the proportion of correctly classified observations:

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + FN + TN}.$$

Accuracy is simple and useful when:

- the classes are reasonably balanced,
- false positives and false negatives have similar costs,
- the decision threshold is already fixed and meaningful.

However, accuracy can be misleading.

Suppose only 1% of observations are positive. A classifier that always predicts negative has 99% accuracy, but it never detects a positive case. In many applications, that classifier is useless.

> **Accuracy is not enough**
>
> Accuracy implicitly treats all mistakes as equally costly.
> When classes are imbalanced or error costs are asymmetric, accuracy can hide the most important behavior of the classifier.

2.1.3 Recall (sensitivity)

Recall is defined as

$$\text{Recall} = \frac{TP}{TP + FN}.$$

Recall answers the question:

> Among all truly positive observations, how many did we detect?

In probability notation,

$$\text{Recall} = P(\hat{Y} = 1 \mid Y = 1).$$

Recall is also called sensitivity, or true positive rate (TPR).

High recall means few false negatives. Low recall means many true positive cases were missed.

Recall and Statistical Power

Recall has a direct connection to hypothesis testing.
Suppose we think of the classification problem as testing

$$H_0 : Y = 0 \quad \text{versus} \quad H_1 : Y = 1.$$

Predicting positive is like rejecting H_0. Predicting negative is like failing to reject H_0.
Under this interpretation:

- a false positive is a Type I error,
- a false negative is a Type II error,
- recall plays a role analogous to statistical power.

Thus

$$\text{Recall} = 1 - \text{Type II error rate}.$$

Recall measures the ability to detect true signals.

2.1.4 Precision

Precision is defined as

$$\text{Precision} = \frac{TP}{TP + FP}.$$

Precision answers the question:

> Among all observations predicted positive, how many are truly positive?

In probability notation,

$$\text{Precision} = P(Y = 1 \mid \hat{Y} = 1).$$

Precision is also called positive predictive value.

High precision means positive predictions are trustworthy. Low precision means many predicted positives are false alarms.

> **Precision vs Recall**
>
> Precision and recall condition on different events.
> Recall conditions on the truth:
>
> $$P(\hat{Y} = 1 \mid Y = 1).$$
>
> Precision conditions on the prediction:
>
> $$P(Y = 1 \mid \hat{Y} = 1).$$
>
> This distinction is very important.
> Recall asks whether we find the positives. Precision asks whether our positive discoveries are reliable. Precision depends strongly on class prevalence.

2.1.5 F1 Score

The F1 score combines precision and recall:

$$F_1 = \frac{2PR}{P + R},$$

where P is precision and R is recall.

Equivalently,

$$F_1 = \frac{2TP}{2TP + FP + FN}.$$

The F1 score is the harmonic mean of precision and recall.

The harmonic mean strongly penalizes imbalance. If precision is high but recall is near zero, then F1 is near zero. If recall is high but precision is near zero, then F1 is also near zero.

F1 is useful when we want a single number that balances precision and recall. However, it assumes precision and recall are equally important. In applications where one type of error is more costly, a different metric or an explicit cost function may be better.

> **Note**
>
> F1 ignores true negatives entirely.
> This can be useful for rare event detection, where the number of true negatives may dominate the dataset and make accuracy misleading.

2.1.6 Recall vs Precision

2.1.6.1 Bayes' Rule and Class Imbalance

Precision and recall are connected by Bayes' rule.

Let $\pi = P(Y = 1)$ be the prevalence of the positive class.

Proposition 2.1.

$$\textit{Precision} = P(Y = 1 \mid \hat{Y} = 1) = \frac{TPR \cdot \pi}{TPR \cdot \pi + FPR(1 - \pi)}.$$

This formula shows why precision can be low for rare events: when π is small, even a modest false positive rate can create many false positives.

Click to expand.

Let

$$TPR = P(\hat{Y} = 1 \mid Y = 1) = \textit{Recall}$$

and

$$FPR = P(\hat{Y} = 1 \mid Y = 0).$$

Then by Bayes' Rule,

$$\begin{aligned}
\textit{Precision} =& P(Y = 1 \mid \hat{Y} = 1) \\
=& \frac{P(\hat{Y} = 1 \mid Y = 1)P(Y = 1)}{P(\hat{Y} = 1 \mid Y = 1)P(Y = 1) + P(\hat{Y} = 1 \mid Y = 0)P(Y = 0)} \\
=& \frac{TPR \cdot \pi}{TPR \cdot \pi + FPR \cdot (1 - \pi)}.
\end{aligned}$$

Example 2.1 (Rare Event Example)**.**

Click to expand.

Suppose there are 10,000 observations:

- 100 positives,
- 9,900 negatives.

Assume the classifier has:

$$TPR = 0.90, \quad FPR = 0.05.$$

Then:

$$TP = 90, \quad FP = 495.$$

The recall is high:

$$\text{Recall} = 0.90.$$

But precision is only

$$\text{Precision} = \frac{90}{90 + 495} \approx 0.154.$$

Most predicted positives are false positives. This is why precision-recall analysis is especially important for imbalanced data.

2.1.6.2 Classification Thresholds

Many classifiers first produce a score or estimated probability:

$$\hat{p}(x) = P(Y = 1 \mid X = x).$$

To make a hard classification, we choose a threshold c:

$$\hat{Y} = \begin{cases} 1, & \hat{p}(x) > c, \\ 0, & \hat{p}(x) \leq c. \end{cases}$$

The default decision threshold is often $c = 0.5$, but this is not always the optimal choice. Changing the threshold changes the confusion matrix, and therefore also changes metrics such as accuracy, precision, recall, false positive rate (FPR), and F1 score.

To better understand the precision–recall tradeoff, let us examine how altering the decision threshold changes the behavior of a classifier.

Decision Threshold	TP	TN	FP	FN	Recall	Precision	Classifier Behavior
Lower threshold	Non-decreasing	Non-increasing	Non-decreasing	Non-increasing	Non-decreasing	May decrease	
Higher threshold	Non-increasing	Non-decreasing	Non-increasing	Non-decreasing	Non-increasing	May increase	

Lowering the threshold classifies more observations as positive. Therefore, both true positives (TP) and false positives (FP) may increase, while false negatives (FN) decrease.

Raising the threshold classifies more observations as negative. Therefore, both true negatives (TN) and false negatives (FN) may increase, while true positives (TP) decrease.

Example 2.2 (Example: Precision Is Not Strictly Monotone)**.**

Click to expand.

Consider a small validation dataset with five observations. The model outputs predicted probabilities $\widehat{P}(Y = 1 \mid X)$, ordered from highest to lowest.

Instance	Predicted Probability	True Label
A	0.95	1
B	0.90	0
C	0.85	1
D	0.80	1
E	0.70	0

As the threshold is raised, fewer observations are classified as positive.

Threshold	Predicted Positives	TP	FP	Precision
0.75	A, B, C, D	3	1	$3/4 = 0.75$
0.82	A, B, C	2	1	$2/3 \approx 0.67$
0.88	A, B	1	1	$1/2 = 0.50$

Threshold	Predicted Positives	TP	FP	Precision
0.92	A	1	0	$1/1 = 1.00$

In this example, raising the threshold first decreases precision because some true positives are removed while the false positive remains. Precision increases only after the false positive is also removed.

This shows that precision can move up or down as the threshold changes, depending on the labels of the observations crossing the threshold.

Thresholds are decisions

- The fitted model may estimate probabilities.
- The threshold determines the action.

Model fitting and decision making are related, but they are not the same thing.

2.1.6.3 Decision Theory and Error Costs

Suppose a false positive has cost C_{FP} and a false negative has cost C_{FN}.

If a model estimates

$$p(x) = P(Y = 1 \mid X = x),$$

then the expected cost of predicting positive is

$$\text{Cost}(1 \mid x) = C_{FP} P(Y = 0 \mid X = x) = C_{FP}(1 - p(x)).$$

The expected cost of predicting negative is

$$\text{Cost}(0 \mid x) = C_{FN} P(Y = 1 \mid X = x) = C_{FN} p(x).$$

We should predict positive when

$$C_{FP}(1 - p(x)) < C_{FN} p(x).$$

Solving this inequality gives

$$p(x) > \frac{C_{FP}}{C_{FP} + C_{FN}}.$$

Thus the optimal threshold is

$$c^* = \frac{C_{FP}}{C_{FP} + C_{FN}}.$$

This threshold minimizes the expected classification cost under the assumed probability model.

If false negatives are very costly, then C_{FN} becomes large, which lowers the threshold. In this case, the classifier predicts the positive class more aggressively in order to reduce missed detections.

Example 2.3 (Medical Screening)**.** In medical screening, missing a serious disease may be much worse than sending a healthy patient for an additional test. Therefore false negatives are costly.

The threshold should often be lower, so the classifier prioritizes recall.

Example 2.4 (Spam Filtering)**.** In spam filtering, putting an important legitimate email into spam may be very costly to the user. Therefore false positives may be more costly.

The threshold should often be higher, so the classifier prioritizes precision.

2.2 Ranking quality

Ranking quality evaluates whether positive observations tend to receive higher scores than negative observations.

This perspective is important when the classification threshold is not fixed or when the main goal is prioritization rather than direct classification. ROC curves, AUC, and precision–recall curves are commonly used to evaluate ranking performance.

2.2.1 ROC

ROC stands for receiver operating characteristic. An ROC curve plots:

$$TPR = \frac{TP}{TP + FN} \quad \text{against} \quad FPR = \frac{FP}{FP + TN}$$

as the probability threshold varies from 0 to 1.

2.2.2 ROC and Hypothesis Testing

ROC curves have a natural hypothesis testing interpretation. As discussed before,

- TPR is power,
- FPR is Type I error rate.

Therefore an ROC curve shows

$$\text{power} \quad \text{versus} \quad \text{Type I error rate}$$

across all possible thresholds.

The ideal classifier reaches the upper-left corner:

$$FPR = 0, \quad TPR = 1.$$

A classifier close to the diagonal line behaves almost like random guessing.

2.2.3 AUC

AUC refers to the Area Under the ROC Curve. It summarizes ranking performance across all possible classification thresholds.

Let $s(X^+)$ be the score for a randomly selected positive observation, and let $s(X^-)$ be the score for a randomly selected negative observation. Then

$$AUC = P\left(s(X^+) > s(X^-)\right),$$

up to small corrections for ties.

Thus AUC has a clean probabilistic interpretation:

> AUC is the probability that a randomly selected positive observation receives a higher score than a randomly selected negative observation.

This explains why AUC is threshold-free. It evaluates ranking quality rather than the quality of a particular classification decision.

Importantly, AUC evaluates relative ordering rather than absolute probability accuracy.

AUC is not calibration

AUC can be excellent even when predicted probabilities are numerically inaccurate. AUC only asks whether positive observations tend to receive higher scores than negative observations. It does not ask whether the predicted probabilities themselves are trustworthy or well calibrated.

2.2.4 Precision-Recall Curve

A precision-recall curve plots precision against recall as the classification threshold changes.

It is especially useful when the positive class is rare. In imbalanced datasets, an ROC curve may look strong because the false positive rate is divided by the large number of true negatives. However, precision directly measures how many predicted positives are actually positive.

Average precision summarizes the precision-recall curve as a single number. Like AUC, it evaluates ranking behavior across thresholds, but it focuses more directly on performance for the positive class.

2.3 Probability Quality

Probability quality asks whether predicted probabilities can be interpreted as actual probabilities.

For example, suppose a model predicts $\hat{p}(x) = 0.8$. If the model is well calibrated, then among many observations with predicted probability near 0.8, roughly 80% should truly belong to the positive class.

Thus probability quality concerns whether predicted probabilities are numerically trustworthy.

Unlike decision quality or ranking quality, probability quality focuses on the accuracy of the probability estimates themselves. Calibration curves, Brier score, and log loss are commonly used to evaluate probability quality.

Note

In many practical machine learning applications, probability quality is less important than classification accuracy or ranking performance.
Therefore, in this course, we will mainly focus on threshold-based classification metrics such as accuracy, precision, recall, and F1 score. This section is included primarily to provide a broader view of classification evaluation.

2.3.1 Calibration Curve

A calibration curve, also called a reliability diagram, is constructed as follows:

1. Divide observations into bins according to predicted probability.
2. For each bin, compute the average predicted probability.
3. Compute the observed positive proportion in each bin.
4. Plot observed proportion against average predicted probability.

For a perfectly calibrated model, the curve follows the diagonal line.

- If the curve lies below the diagonal, the model is overconfident.
- If the curve lies above the diagonal, the model is underconfident.

2.4 Model Tuning versus Threshold Tuning

In classification, model fitting and decision making should be viewed as separate problems.

First, the model estimates a score or probability:

$$\hat{p}(x) \approx \Pr(Y = 1 \mid X = x).$$

Second, we choose a threshold to convert this probability into an action.

Model tuning affects the quality of $\hat{p}(x)$ itself. This is where ideas such as bias, variance, regularization, and model complexity matter. A very flexible classifier may capture nonlinear structure, but its probability estimates may be unstable or poorly calibrated.

Threshold tuning does not change $\hat{p}(x)$. It only changes the decision rule built on top of the fitted probabilities. Lower thresholds usually increase recall and false positives; higher thresholds usually increase precision and reduce false positives.

Thus:

- model tuning controls the fitted scores or probabilities;
- threshold tuning controls the action taken from those scores;
- calibration checks whether the probabilities are numerically trustworthy.

A good classifier therefore involves not only learning accurate scores, but also choosing appropriate decision rules for the application.

2.5 Python Example: Breast Cancer Classification

We now use the breast cancer dataset from `sklearn`. The goal is to classify tumors as malignant or benign. This example shows how to compute threshold-based metrics, ROC and PR curves, and calibration curves.

In this example, please mainly focus on the evaluation metrics. Other part like modeling, splitting, etc. will be introduced in later sections.

2.5.1 Load the dataset

First load the data and split it into training and test sets. By default, the dataset uses `0` for malignant and `1` for benign. Since many classification metrics interpret label `1` as the positive class, we redefine malignant as `1`. Therefore we need to modify the `y` value.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

cancer = load_breast_cancer(as_frame=True)

X = cancer.data
y_original = cancer.target
y = 1 - y_original

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=1
)
```

We may want to see the distribution of the test set.

```
import pandas as pd
pd.Series(y_test).value_counts().sort_index()
```

```
target
0    107
1     64
Name: count, dtype: int64
```

2.5.2 Fit models

We fit logistic regression model as an example, which is a typical strong baseline model.

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

logit = make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=5000),
)

logit.fit(X_train, y_train)

logit_prob = logit.predict_proba(X_test)[:, 1]
```

Note that `logit.predict_proba(X_test)` returns 2 columns:

- column 0: $\Pr(Y = 0 \mid X)$,
- column 1: $\Pr(Y = 1 \mid X)$.

Since we want the predicted probability for the positive class (`y=1`), we select the second column.

2.5.3 Confusion Matrix and Basic Metrics

All decision quality metrics can be computed using functions from `sklearn.metrics`. They are used to compare two list-like objects, while the true labels are passed first and the predicted labels second.

By default, `predict()` in `sklearn` uses threshold 0.5 for binary classification.

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

y_pred = logit.predict(X_test)

acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred)
rec = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
```

We can also compute the confusion matrix and extract the values of TP, TN, FP, and FN from it.

```
from sklearn.metrics import confusion_matrix

confusion_matrix(y_test, y_pred)
```

```
array([[106,   1],
       [  6,  58]])
```

```
TN, FP, FN, TP = confusion_matrix(y_test, y_pred).ravel()
```

They are listed in this particular order since 0 stands for negative and 1 stands for positive in this example.

2.5.4 Modifying Thresholds

The default model use 0.5 as the threshold in `sklearn`. If you want to change the threshold, you may manually write the code.

```
threshold = 0.3
logit_prob = logit.predict_proba(X_test)[:, 1]
y_pred = (logit_prob >= threshold).astype(int)
```

You may also use `FixedThresholdClassifier` from `sklearn.model_selection` when you want the threshold to be part of the estimator workflow.

```
from sklearn.model_selection import FixedThresholdClassifier

threshold = 0.3
model_with_threshold = FixedThresholdClassifier(
    make_pipeline(
        StandardScaler(),
        LogisticRegression(max_iter=5000),
    ),
    threshold=threshold,
)

model_with_threshold.fit(X_train, y_train)
y_pred_wrapper = model_with_threshold.predict(X_test)
```

> **Note**
>
> `FixedThresholdClassifier` is useful when the threshold is fixed in advance. However, there might be cases that you want to adjust the threshold according to the dataset. In this case you may want to use `TunedThresholdClassifierCV`. We will talk about it in details later in the Logistic regression section.

2.5.5 Tuning Thresholds

In order to make the tuning process easier, we build an interface to quickly display the results.

```
def classification_summary(y_true, prob, threshold=0.5):
    pred = (prob >= threshold).astype(int)
    TN, FP, FN, TP = confusion_matrix(y_true, pred).ravel()

    return {
        "threshold": threshold,
        "TP": TP,
        "FP": FP,
        "FN": FN,
        "TN": TN,
        "accuracy": accuracy_score(y_true, pred),
        "precision": precision_score(y_true, pred),
        "recall": recall_score(y_true, pred),
        "f1": f1_score(y_true, pred),
    }
```

Now examine how metrics change as the threshold changes. To save space I only show the first few rows of the table.

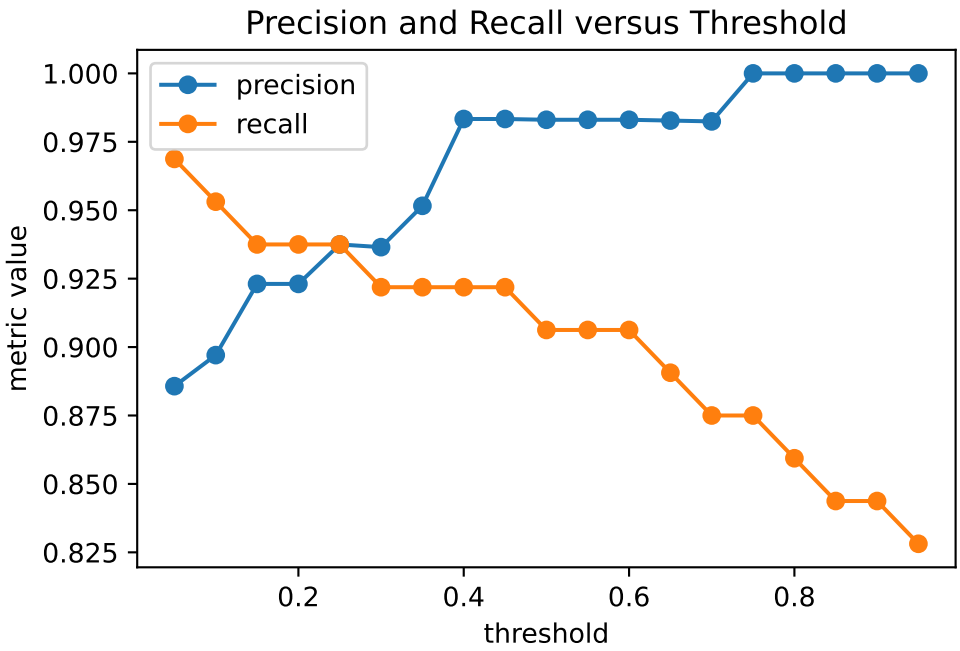
```
import numpy as np
import pandas as pd

thresholds = np.linspace(0.05, 0.95, 19)
threshold_results = []

for threshold in thresholds:
    row = classification_summary(y_test, logit_prob, threshold=threshold)
    threshold_results.append(row)

threshold_df = pd.DataFrame(threshold_results)
```

```
threshold_df.head()
```

	threshold	TP	FP	FN	TN	accuracy	precision	recall	f1
0	0.05	62	8	2	99	0.941520	0.885714	0.968750	0.925373
1	0.10	61	7	3	100	0.941520	0.897059	0.953125	0.924242
2	0.15	60	5	4	102	0.947368	0.923077	0.937500	0.930233
3	0.20	60	5	4	102	0.947368	0.923077	0.937500	0.930233
4	0.25	60	4	4	103	0.953216	0.937500	0.937500	0.937500

Plot precision and recall as functions of the threshold.

```
import matplotlib.pyplot as plt

plt.plot(threshold_df["threshold"], threshold_df["precision"], marker="o", label="precision")
plt.plot(threshold_df["threshold"], threshold_df["recall"], marker="o", label="recall")
plt.xlabel("threshold")
plt.ylabel("metric value")
plt.title("Precision and Recall versus Threshold")
plt.legend()
```



As the threshold increases, the classifier becomes more conservative in predicting the positive class. This often reduces false positives and increases precision, but may also increase false negatives and reduce recall.

2.5.6 Choosing a Threshold from Costs

In a medical setting, false negatives may be especially important because they correspond to missed malignant cases. Suppose a false negative is 5 times as costly as a false positive:

$$C_{FN} = 5, \quad C_{FP} = 1.$$

The decision-theoretic threshold is

$$c^* = \frac{C_{FP}}{C_{FP} + C_{FN}} = \frac{1}{1+5} \approx 0.167.$$

Then the corresponding result is

```
cost_fp = 1
cost_fn = 5

cost_threshold = cost_fp / (cost_fp + cost_fn)

classification_summary(
    y_test,
    logit_prob,
    threshold=cost_threshold,
)
```

```
{'threshold': 0.16666666666666666,
 'TP': np.int64(60),
 'FP': np.int64(5),
 'FN': np.int64(4),
 'TN': np.int64(102),
 'accuracy': 0.9473684210526315,
 'precision': 0.9230769230769231,
 'recall': 0.9375,
 'f1': 0.9302325581395349}
```

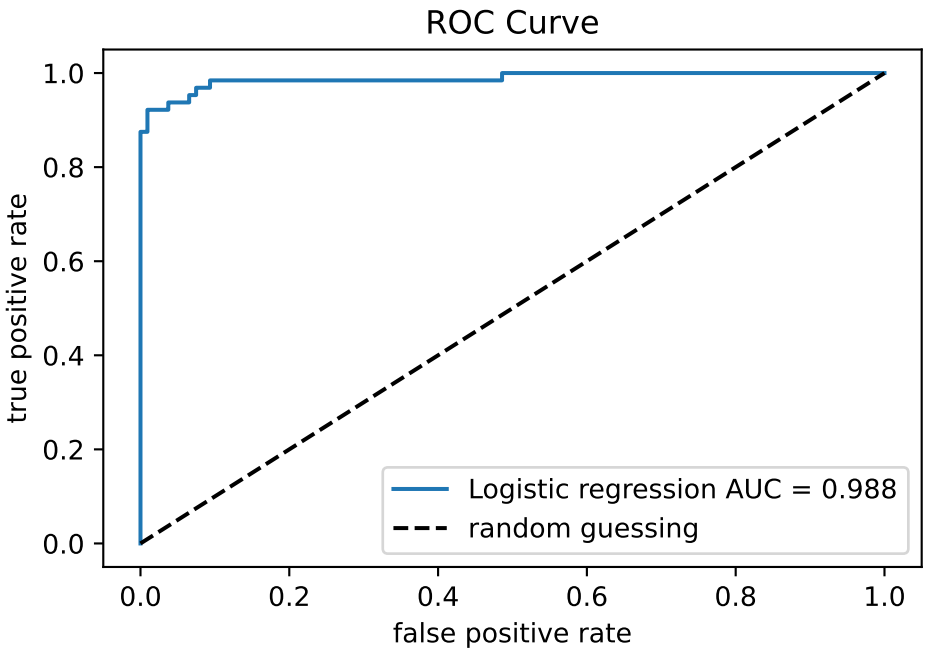
This lower threshold prioritizes recall. It predicts malignant more aggressively because missing a malignant tumor is assumed to be more costly. Note that this threshold formula assumes that the predicted probabilities are reasonably calibrated so that they can be interpreted as approximate event probabilities.

2.5.7 ROC Curve and AUC

Unlike accuracy or precision, ROC analysis does not depend on a single fixed threshold. Instead, it studies model performance across all possible thresholds using the predicted probabilities or scores.

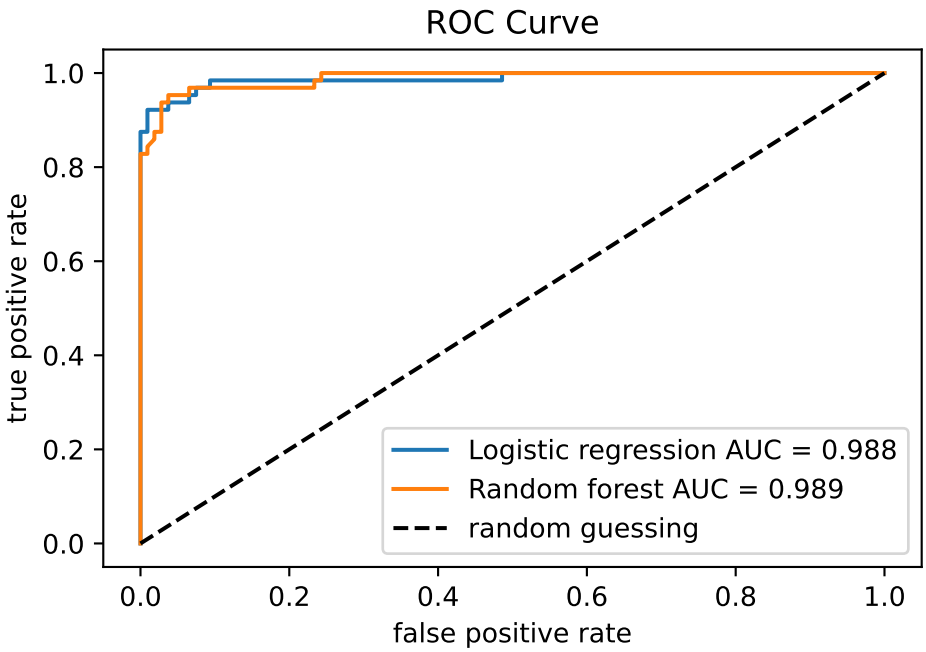
ROC curve and AUC can be computed using `roc_curve` and `roc_auc_score` from `sklearn.metrics`.

```
from sklearn.metrics import roc_curve, roc_auc_score

logit_fpr, logit_tpr, _ = roc_curve(y_test, logit_prob)
logit_auc = roc_auc_score(y_test, logit_prob)

plt.plot(logit_fpr, logit_tpr, label=f"Logistic regression AUC = {logit_auc:.3f}")
plt.plot([0, 1], [0, 1], "k--", label="random guessing")
plt.xlabel("false positive rate")
plt.ylabel("true positive rate")
plt.title("ROC Curve")
plt.legend()
plt.show()
```



We may fit another model and draw their ROC curve and AUC together in one plot to compare them.

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=500, max_features="sqrt", random_state=1, n_jobs=-1
)

rf.fit(X_train, y_train)
rf_prob = rf.predict_proba(X_test)[:, 1]

rf_fpr, rf_tpr, _ = roc_curve(y_test, rf_prob)
rf_auc = roc_auc_score(y_test, rf_prob)

plt.plot(logit_fpr, logit_tpr, label=f"Logistic regression AUC = {logit_auc:.3f}")
plt.plot(rf_fpr, rf_tpr, label=f"Random forest AUC = {rf_auc:.3f}")
plt.plot([0, 1], [0, 1], "k--", label="random guessing")
plt.xlabel("false positive rate")
plt.ylabel("true positive rate")
plt.title("ROC Curve")
plt.legend()
```

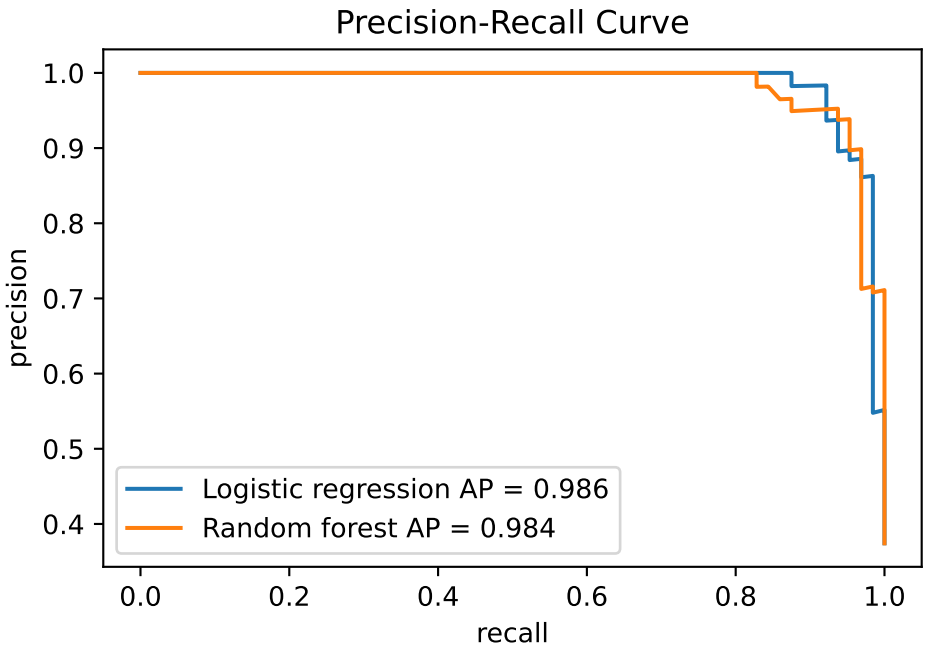


The ROC curve summarizes ranking performance across thresholds. AUC may be interpreted as the probability that a randomly chosen positive observation receives a higher score than a randomly chosen negative observation. Then a larger AUC means the model more often ranks malignant cases above benign cases.

2.5.8 Precision-Recall Curve

The precision-recall curve can be produced in a similar way.

```
from sklearn.metrics import precision_recall_curve, average_precision_score

logit_precision, logit_recall, _ = precision_recall_curve(y_test, logit_prob)
rf_precision, rf_recall, _ = precision_recall_curve(y_test, rf_prob)

logit_ap = average_precision_score(y_test, logit_prob)
rf_ap = average_precision_score(y_test, rf_prob)

plt.plot(logit_recall, logit_precision, label=f"Logistic regression AP = {logit_ap:.3f}")
plt.plot(rf_recall, rf_precision, label=f"Random forest AP = {rf_ap:.3f}")
plt.xlabel("recall")
plt.ylabel("precision")
plt.title("Precision-Recall Curve")
plt.legend()
```



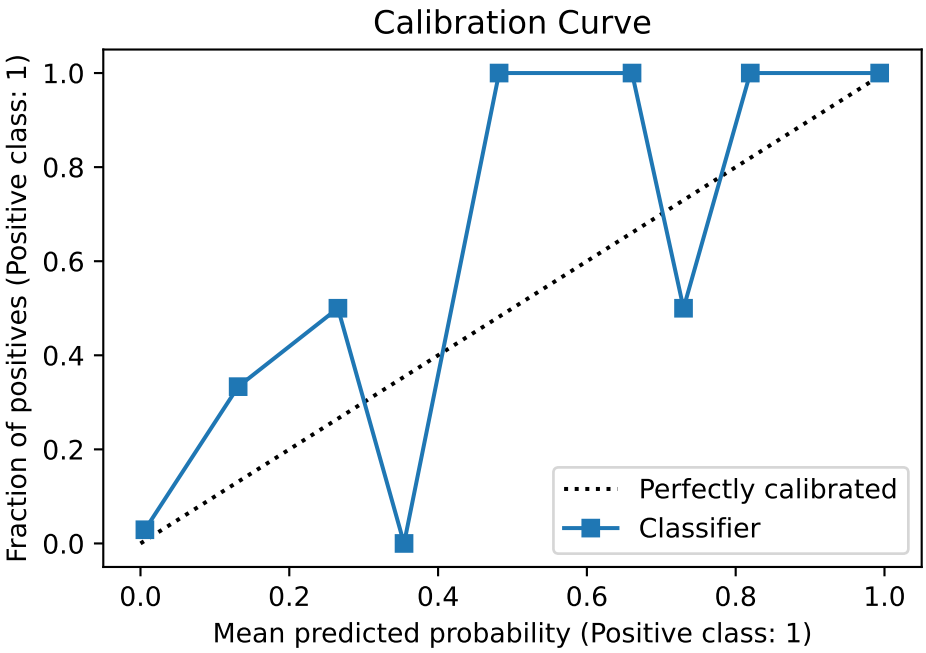
For highly imbalanced datasets, precision–recall curves are often more informative than ROC curves because ROC curves may appear overly optimistic when the negative class dominates.

2.5.9 Calibration Curves

Calibration focuses on probability quality rather than threshold-based decision quality or ranking quality. Calibration curves can be produced using `CalibrationDisplay` from `sklearn.calibration`.

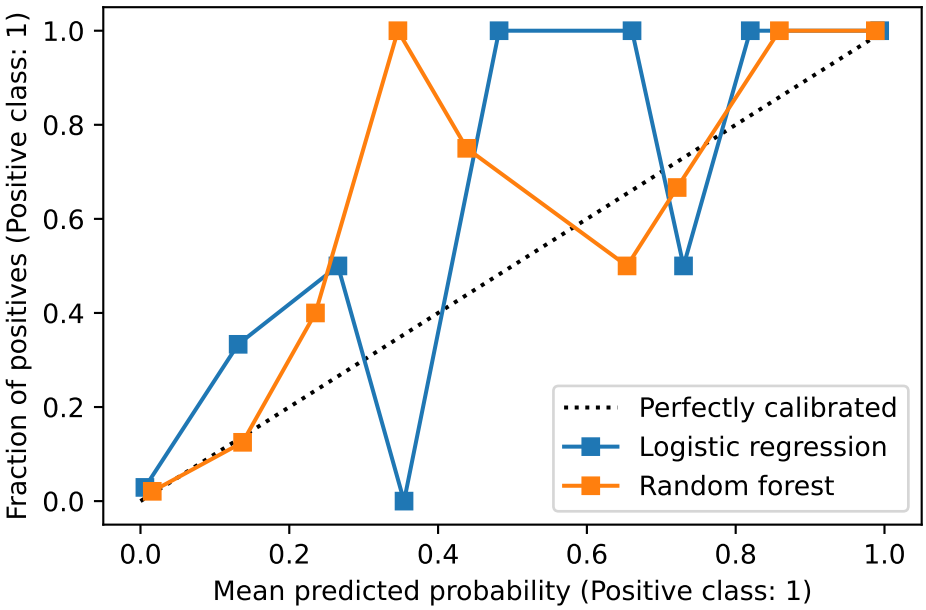
```
from sklearn.calibration import CalibrationDisplay

CalibrationDisplay.from_predictions(y_test, logit_prob, n_bins=10)
plt.title("Calibration Curve")
```

```
Text(0.5, 1.0, 'Calibration Curve')
```



This is a packaged function provided by `sklearn`. If we want to plot multiple curves together in one plot, we can put the axis in the argument of this function.

```
fig, ax = plt.subplots()

CalibrationDisplay.from_predictions(
    y_test, logit_prob, n_bins=10, name="Logistic regression", ax=ax
)

CalibrationDisplay.from_predictions(
    y_test, rf_prob, n_bins=10, name="Random forest", ax=ax
)
```



The calibration curve compares predicted probabilities with observed frequencies.

Calibration curves diagnose probability quality; they do not by themselves recalibrate the model. Recalibration methods such as Platt scaling or isotonic regression can change the probability scale, but they do not necessarily improve AUC because AUC mainly depends on ranking.

A Python IDE Setup

There are many ways to set up a Python development environment, and each has its own advantages and disadvantages. In this tutorial, we will cover one of the most popular current setups: VS Code + `uv`.

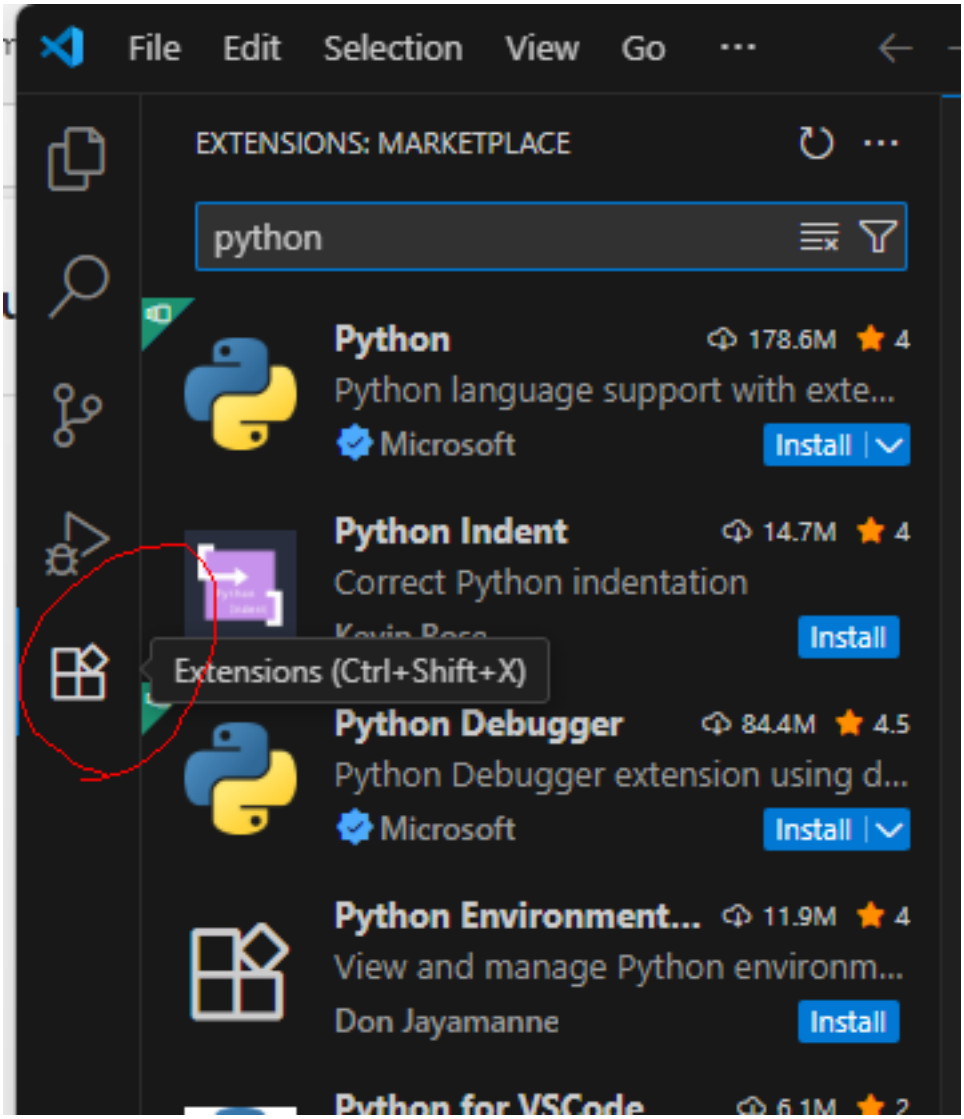
For now, we will focus only on the practical setup steps. The underlying concepts will be discussed in the later appendices.

This tutorial uses Windows 11. If you use another operating system, the steps are very similar, but you may need to make some minor but straightforward modifications. If you encounter major issues, please contact me.

A.1 VS Code

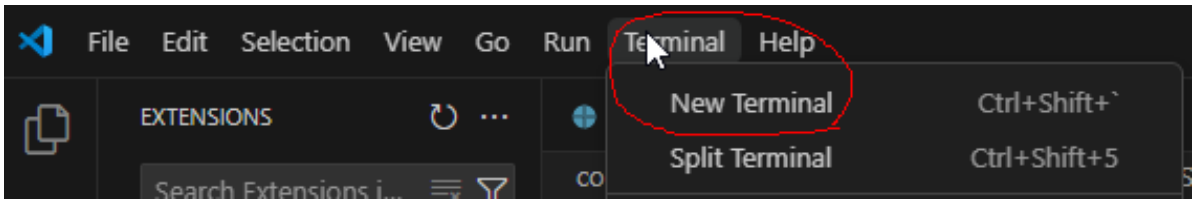
We choose VS Code as our IDE for Python. The installation is straightforward starting from the Official website. After installation, we need to do a few configurations to make the experience better.

1. For development of Python, the essential plugins are the Python plugin and Jupyter plugin. You only need to install the two main ones, and all related plugins will be installed automatically.

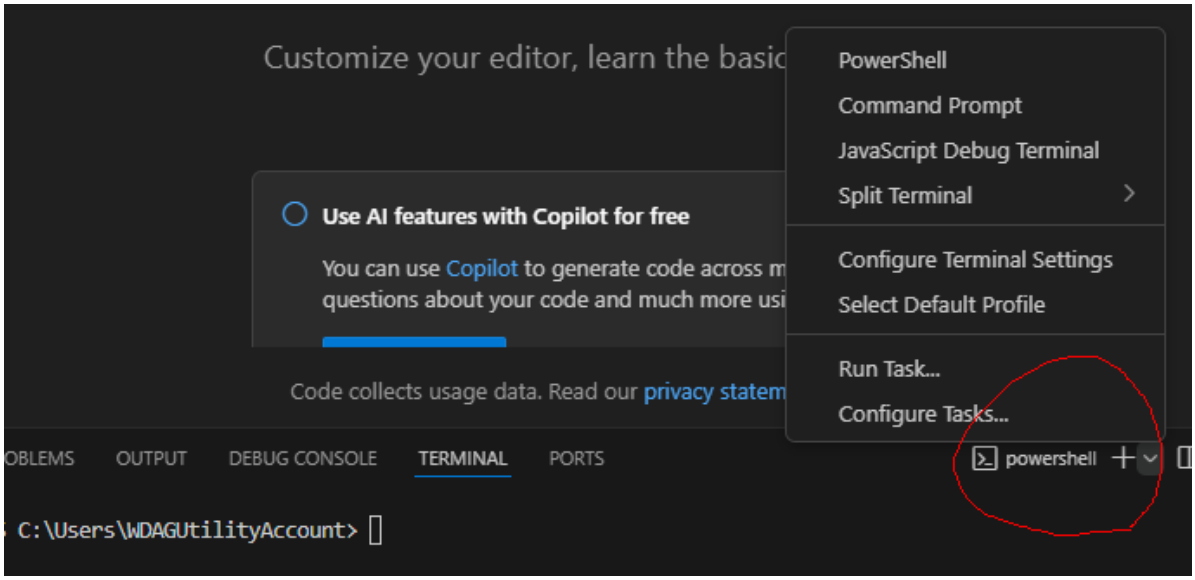


You may either search the extensions directly from VS Code extension tab (which is usually located on the left side bar), or you may install it from the link python and jupyter.

2. Sometimes you may need to use the terminal inside VS Code. You may start the terminal from top menu (whose hotkey is by default `ctrl+shift+`).



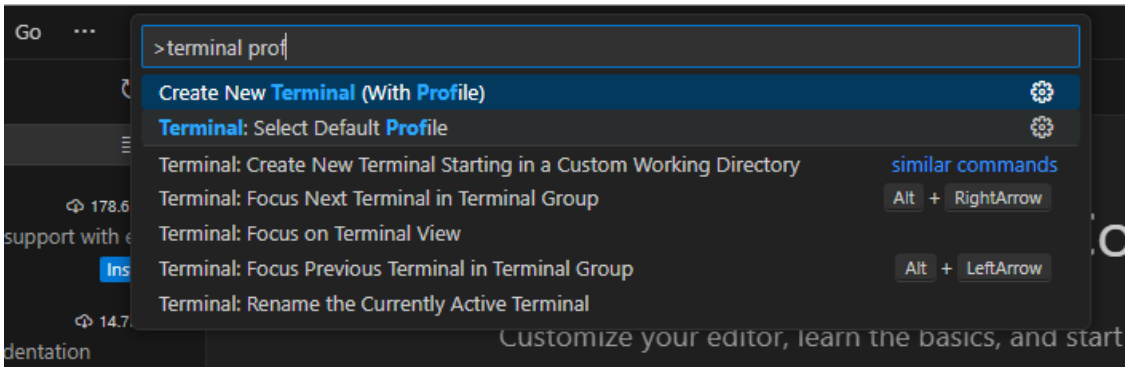
Note that there are many different choices of terminals. After you start the first default terminal, you may start any new terminals using the small `+` and the `v` mark on the right.



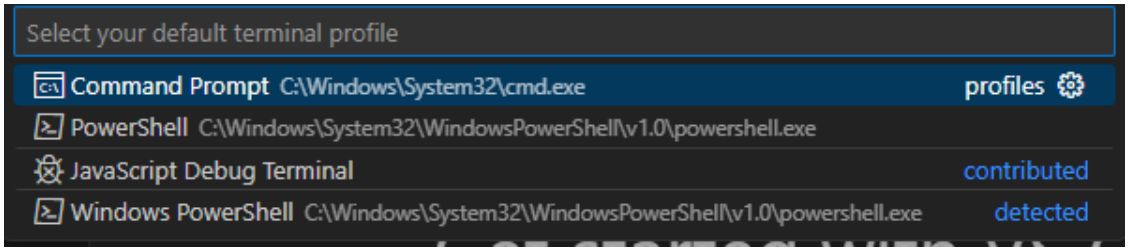
Change Terminal default profile

In Windows the default terminal is Powershell when you first install VS Code. In the past it had bugs with `conda` (and I don't know whether it is fixed now). In order to avoid headaches due to the bugs, we simply switch to other terminals, like Command Prompt. You may change the default terminal by

1. Press `Ctrl+Shift+P` or `F1` to open the top prompt menu.
2. Find the `Terminal: Select Default Profile` command.



3. Select the desired terminal as the default one.



A.2 Python Virtual Environment installation

There are many tools available for managing Python environments and packages. As of Summer 2026, `uv` has become one of the most popular choices.

`uv` is an all-in-one tool for managing Python installations, virtual environments, and project dependencies. With `uv`, you do not need to install Python separately, as it can download and manage Python versions for you. You may visit the official documentation to install `uv`.

Windows headache

There is a possibility that Smart App Control (SAC) or other Application Control policies in Windows 11 may block the execution of `uv.exe`. In this case, you may see a message similar to

```
'C:\Users\WDAGUtilityAccount\.local\bin\uv.exe'
was blocked by your organization's Device Guard policy.
Contact your support person for more info.
```

This may occur because Windows does not yet consider the executable sufficiently trusted under its application-control policies.
If this happens, note that the exact cause and solution can vary across machines, Windows versions, and security configurations. In addition, both Windows and `uv` are evolving rapidly, so solutions that work today may change in the future.
You are encouraged to search for the solution that best fits your situation. For this course, one workaround is to use WSL (see Section A.4).

After installation (and you may need to restart VS Code or open a new terminal window so that the `uv` command becomes available), you can use the following commands to verify that `uv` is working correctly.

Command	Purpose	Example Output
`uv --version`	Display the installed `uv` version. Useful for verifying that `uv` is installed correctly.	`uv 0.11.17`
`uv self update`	Update `uv` itself to the latest available version. Similar to updating package managers such as `pip` or `cargo`.	`Upgraded uv from v0.11.4 to v0.11.17!`
`uv python list`	Show all Python versions available locally and those that can be installed through `uv`.	Lists installed and downloadable Python versions.

A.2.1 Install the first Python

Since VS Code often expects to discover at least one Python interpreter before it can fully initialize its Python and Jupyter services, we first install a Python version managed by `uv`.

```
uv python install
```

This Python installation is managed entirely by `uv`; it is not intended to be used directly for course work. Instead, it serves as a bootstrap interpreter that allows VS Code to detect Python correctly and manage project-specific virtual environments more reliably. You can verify the installed Python versions using `uv python list` as mentioned above.

If you skip this step and directly jump into the virtual environment, there is a possibility that Python in VS Code doesn't work well.

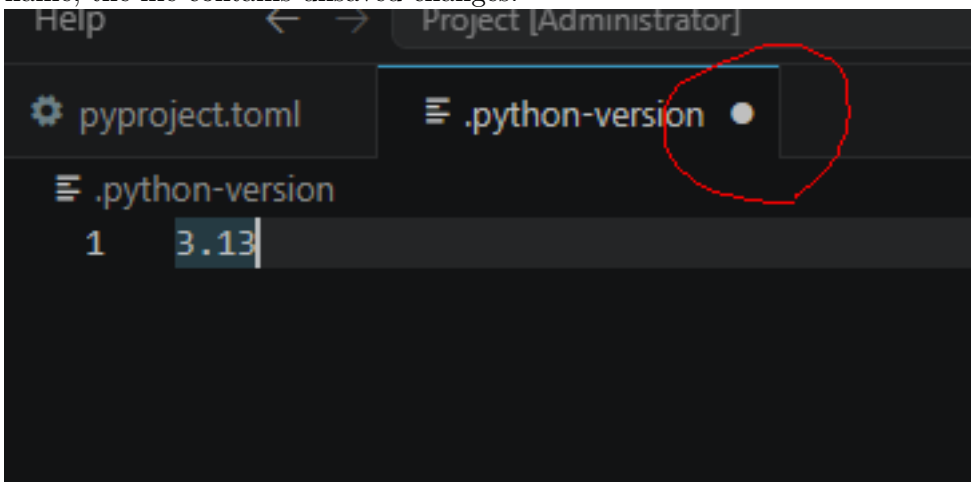
A.2.2 Setup the environment for the course

To set up the environment for this course, we provide the toml file and python version file for this course.

```
[project]
name = "STAT432"
version = "0.1.0"
requires-python = ">=3.13"
dependencies = [
    "jupyter>=1.1.1",
    "matplotlib>=3.10.9",
```

```
    "openpyxl>=3.1.5",
    "pandas>=3.0.2",
    "scikit-learn>=1.8.0",
    "seaborn>=0.13.2",
    "statsmodels>=0.14.6",
    "xgboost>=3.2.0",
]
```

The python version file only contains the version number `3.13` inside since this is the version of Python I use when I wrote the notes. You may either download the file and put it inside the folder or create a file with the name `.python-version` and write `3.13` inside.

> **Autosave**
>
> Before proceeding, make sure the file is saved.
> By default, VS Code does not enable Auto Save. If you see a small dot next to the file name, the file contains unsaved changes.
>
> 
>
>
> Many beginner problems occur because they modify a file but forget to save it before running commands.

1. Create a new folder. The folder name is irrelevant. Put the files `pyproject.toml` and `.python-version` in the folder.
2. Open a terminal (no matter whether within VS Code or not) and enter the folder. Use the command `uv sync`.

After the command finishes, `uv` creates a virtual environment named `.venv` and installs all required packages specified in `pyproject.toml`. The resulting environment should closely match the one used to prepare these notes.

Now it is possible to start using the virtual environment for the course. In case that you need to modify the virtual environment, please continue the read the following optional part.

Jupyter kernels and modifying venv

A Jupyter kernel is the Python environment used by a notebook. When a notebook cell is executed, the code runs inside the selected kernel. For this reason, it is important to connect the notebook to the kernel associated with your course virtual environment.
Modern versions of VS Code can often detect a virtual environment automatically and use it directly as a notebook kernel without requiring you to manually register a Jupyter kernel using `ipykernel` install. Therefore if everything works well the actions before this note is enough.
However, there are situations in which you may need to create a dedicated Jupyter kernel manually. In addition, you may occasionally need to modify the virtual environment by installing additional packages. Therefore, it is useful to understand these operations.

1. Activate the virvutal environment.

```
# Windows
.venv\Scripts\activate

# Linux / macOS
source .venv/bin/activate
```

2. After activation, your terminal prompt will typically change to something like indicating that the virtual environment is active:

```
# Windows
(stat432) C:\Users\yourname\project>

# Linux / macOS
(stat432) user@computer:~/project$
```

Note that the name `stat432` is indicated in the `pyproject.toml` that `name = "STAT432"`. It doesn't do anything special if you don't publish this environment. It is different from the jupyter kernel name.

3. You may use `where python` (Windows) or `which python` (Linux / macOS) to see whether the correct python version is activated. The path should be `.venv` in the current folder.
4. To install an additional package and update the project configuration, use

```
uv add <package name>
```

More commands can be found in uv official document.

5. If you need to create a dedicated Jupyter kernel, run

```
python -m ipykernel install --user --name stats --display-name "(stats)"
```

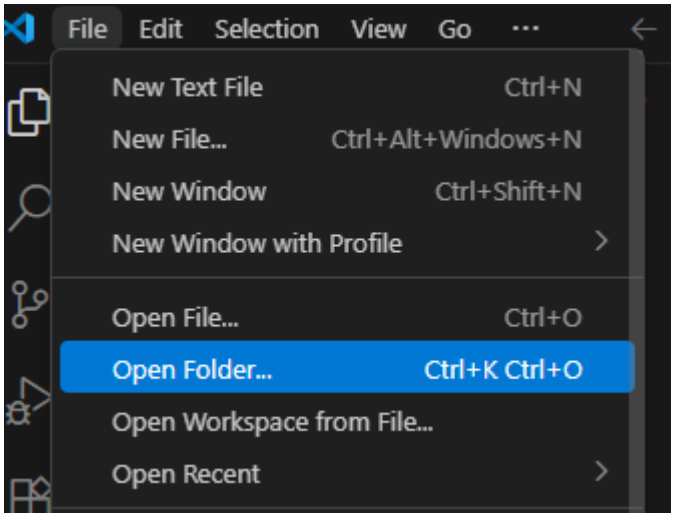
Note that `--name stats` specifies the kernel identifier. In this example the identifier is `stats`, but you may choose any name that is easy for you to recognize. Later, when selecting a kernel in Jupyter or VS Code, this identifier is used internally to distinguish one kernel from another, so it should be unique on your system.
The option `--display-name "(stats)"` specifies the name displayed in Jupyter and VS Code when selecting a kernel. Unlike `--name`, the display name does not need to be unique, although choosing a descriptive name is recommended.

A.3 Back to VS Code and start Jupyter notebook

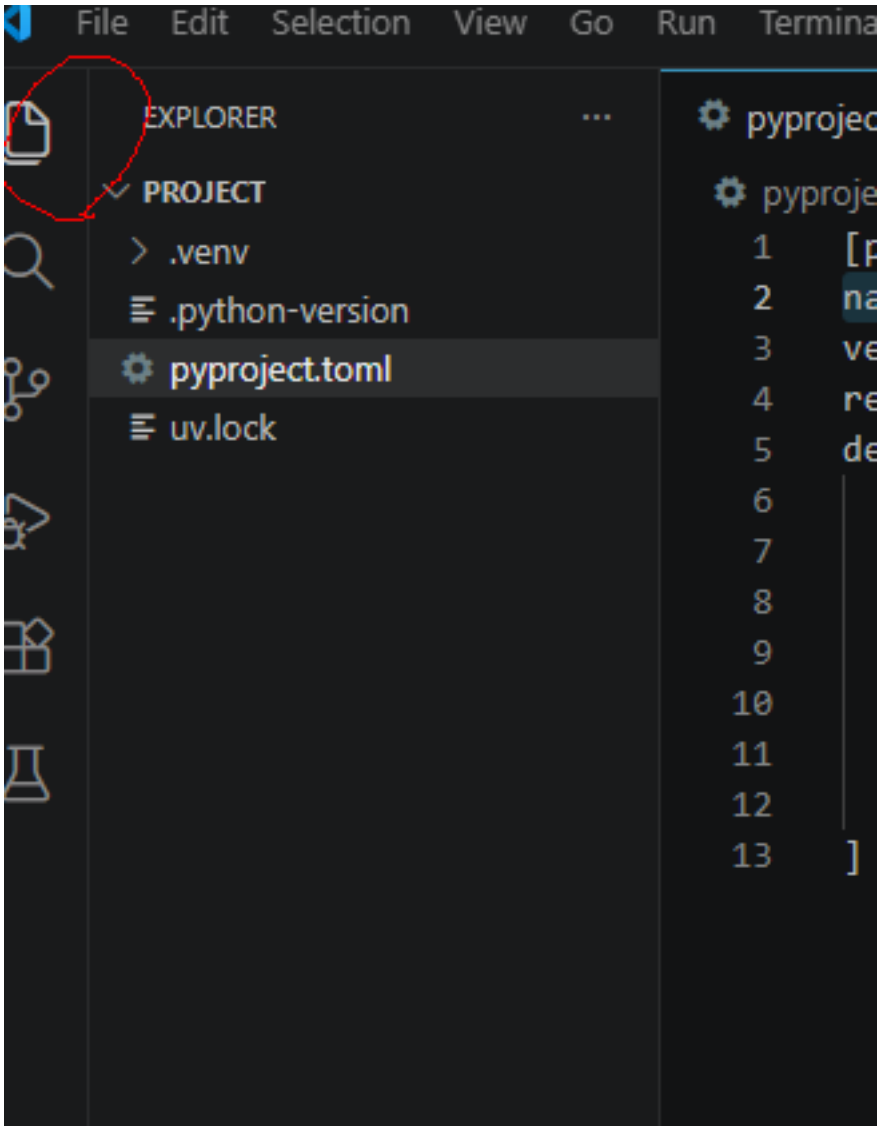
After Python is installed and virtual environment is set up, we could start to do a Hello World project.

The way VS Code organizes projects is through folders by default. So we first start a new folder (called the working folder) and all project related files should be put inside this folder. (There are ways to work with files outside the working folder, but you don't need it in most cases.)

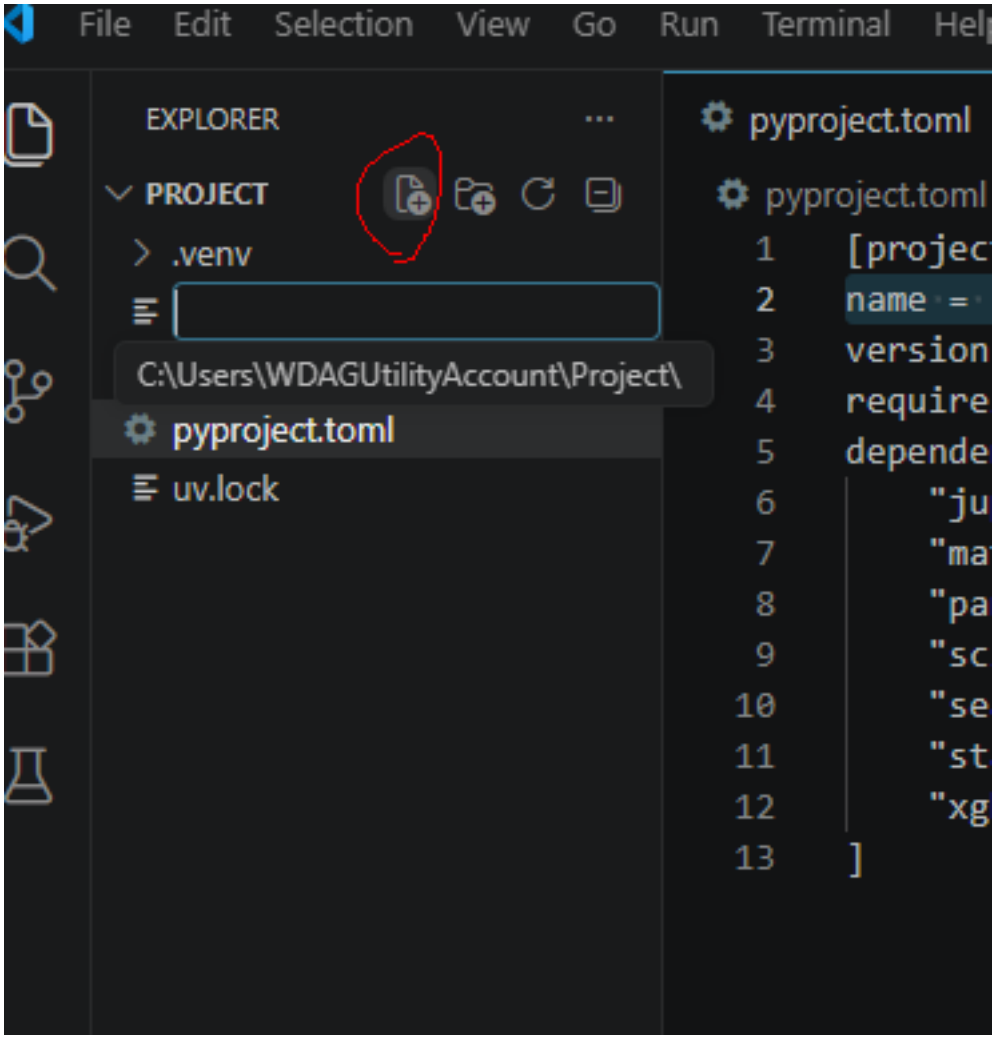
Since we already set up a folder (with a virtual environment), we open it as our working folder.



The folder already contains the project files and the environment files. You may see them from the file explorer.

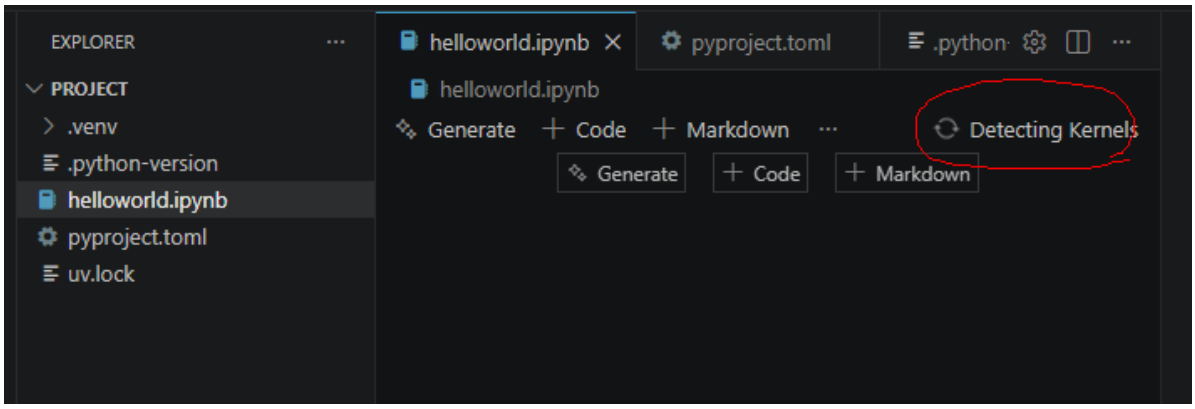


Then you may create new files/folders or copy files/folders into this working folder. For demonstration a new file called `helloworld.ipynb` is created.

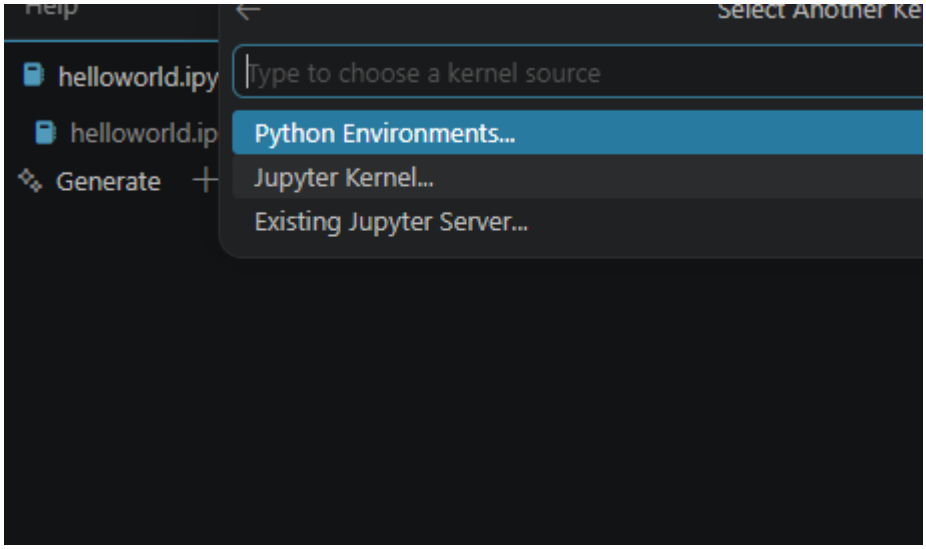


`ipynb` means this is a Jupyter notebook file (which is short for `interactive Python notebook`). This is the format for this course's homework assignment.

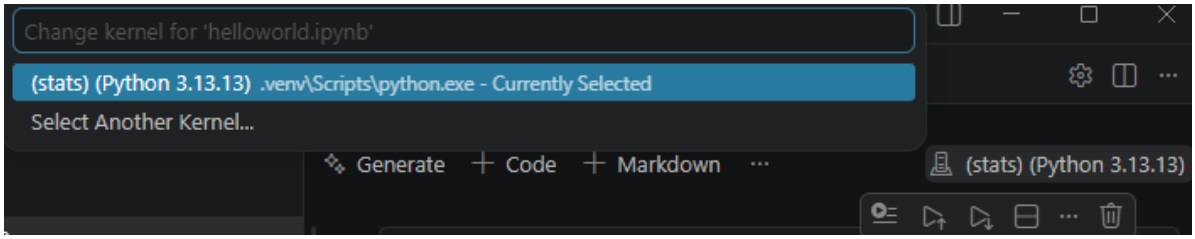
This is the appearance of an empty notebook file.



We attach the kernel we just created to this notebook. Click the `Select Kernel` / `Detecting Kernels` on the right upper corner and select the environment we want.

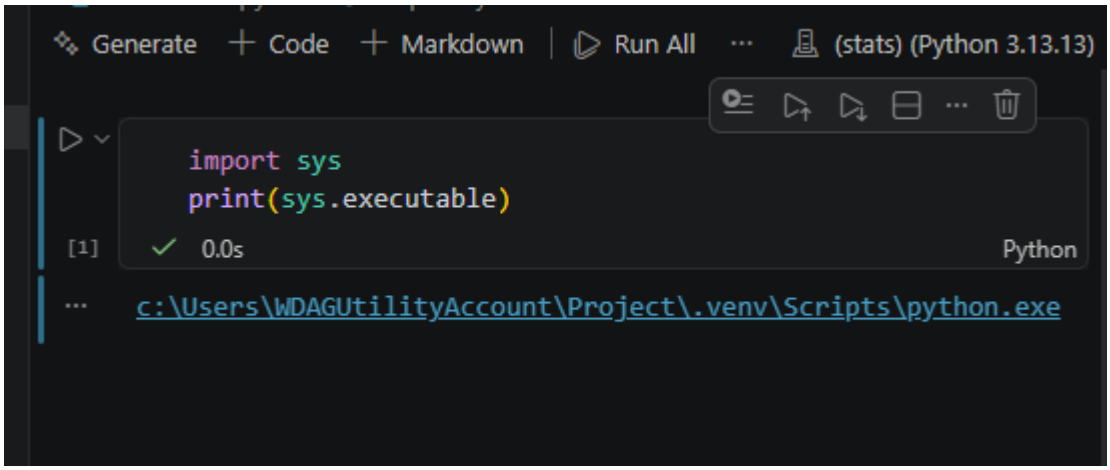


- If you created a new Jupyter kernel, you can find it from `Jupyter kernel...`.
- If not, you can find your virtual environement in the working folder from `Python Environments...`.



After the kernel is selected, we could start using the notebook. I prefer to use the following code as hello world. It can also tell us whether the correct python is used.

```
import sys
print(sys.executable)
```



A.3.1 Output to html and pdf

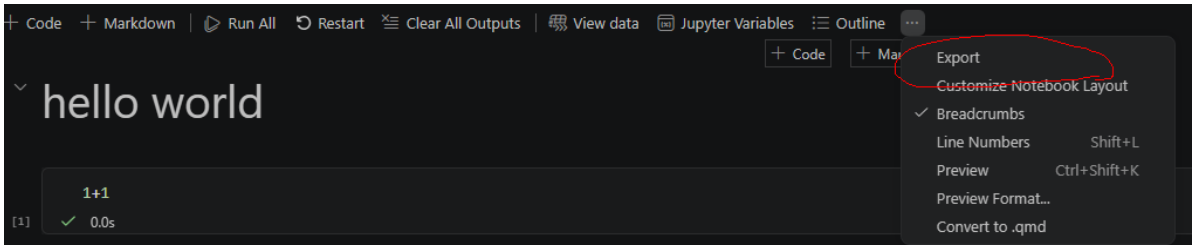
After finishing your Jupyter Notebook, you need to generate a PDF version for assignment submission.

Since we do not want to spend too much time on document formatting, we will use a simple workflow. Although there are many more sophisticated tools available (such as Pandoc, Quarto, and LaTeX-based workflows), the approach below is completely sufficient for this course.

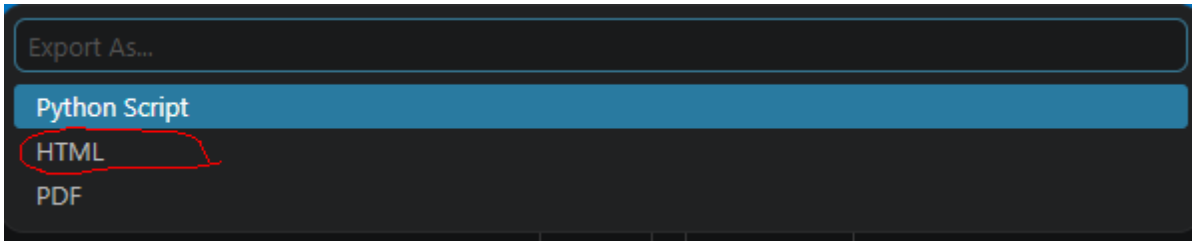
Our workflow is:

1. Export the notebook to HTML using VS Code.
2. Open the HTML file in a web browser.
3. Print the page to a PDF file.

First, locate the notebook toolbar and click the `...` button on the right side. From the expanded menu, select `Export`.



Then choose `HTML`.



This will create an HTML file. Open the file in your preferred web browser and print it to PDF. Most modern browsers (such as Chrome, Edge, and Firefox) support printing directly to PDF.

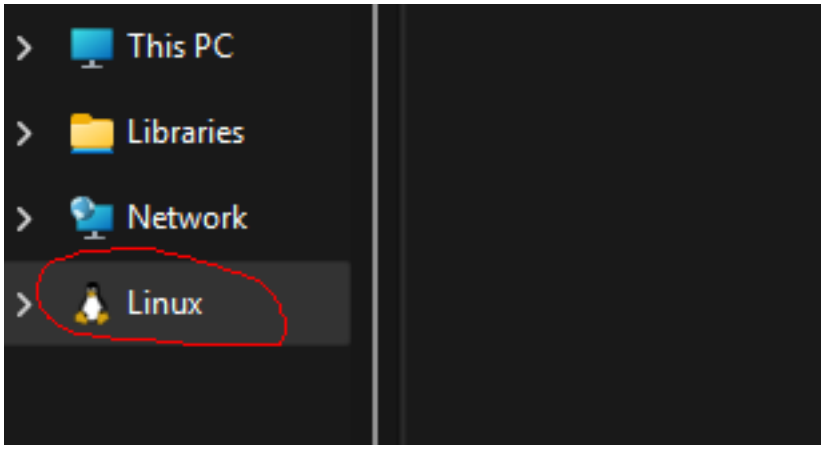
For assignment submission, submit both the PDF file and the notebook file. Grading will be based primarily on the PDF file, while the notebook file serves as a backup source in case I need to inspect the code or verify specific parts of the submission.

A.4 WSL

WSL stands for Windows subsystem for Linux. It allows you to run a Linux environment directly inside Windows without installing a separate operating system.

To install WSL, please follow the official Microsoft guide.

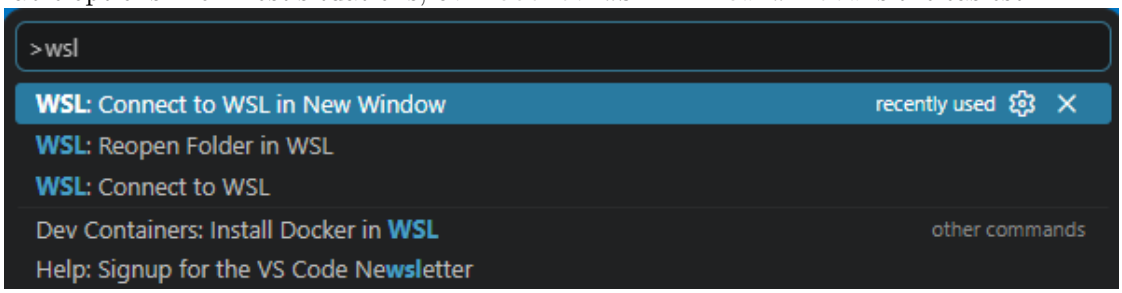
After installation, you will have a Linux system (usually Ubuntu) running alongside Windows. The Linux file system is separate from the Windows file system, although you can access it through File Explorer.



VS Code provides excellent support for WSL through the Remote WSL extension. To open a WSL window, you may:

- Click the button in the lower-left corner of VS Code 

- Or open the Command Palette (`Ctrl+Shift+P` or `F1`), search for `WSL`, and select one of the available options. For most situations, `Connect to WSL in New Window` is the easiest choice.



Once connected to WSL, you are working inside a Linux environment rather than Windows.

Because WSL is a separate system, you will need to:

- Open your project folder again.
- Install Python and other tools inside WSL.
- Create virtual environments inside WSL.
- Install packages inside WSL.

You may follow the same instructions presented earlier to install `uv` and set up your Python environment. The overall process is the same, although the commands will be the Linux versions rather than the Windows versions.

Note that if you install `uv` in Windows, it is not automatically available inside WSL. Similarly, software installed inside WSL is generally not available in Windows.

In practice, it is usually easiest to keep an entire project inside WSL and perform all Python-related work there.

> **Tip**
>
> Avoid storing virtual environments inside the Windows file system when working primarily in WSL. For best performance and compatibility, create your project folders inside your Linux home directory (for example, `~/Project`) rather than in mounted Windows directories (for example, `/mnt/c/...`).

B Jupyter Notebook and Markdown

One of the best features of a Jupyter Notebook is that it can combine code, narrative text, and output in a single file.

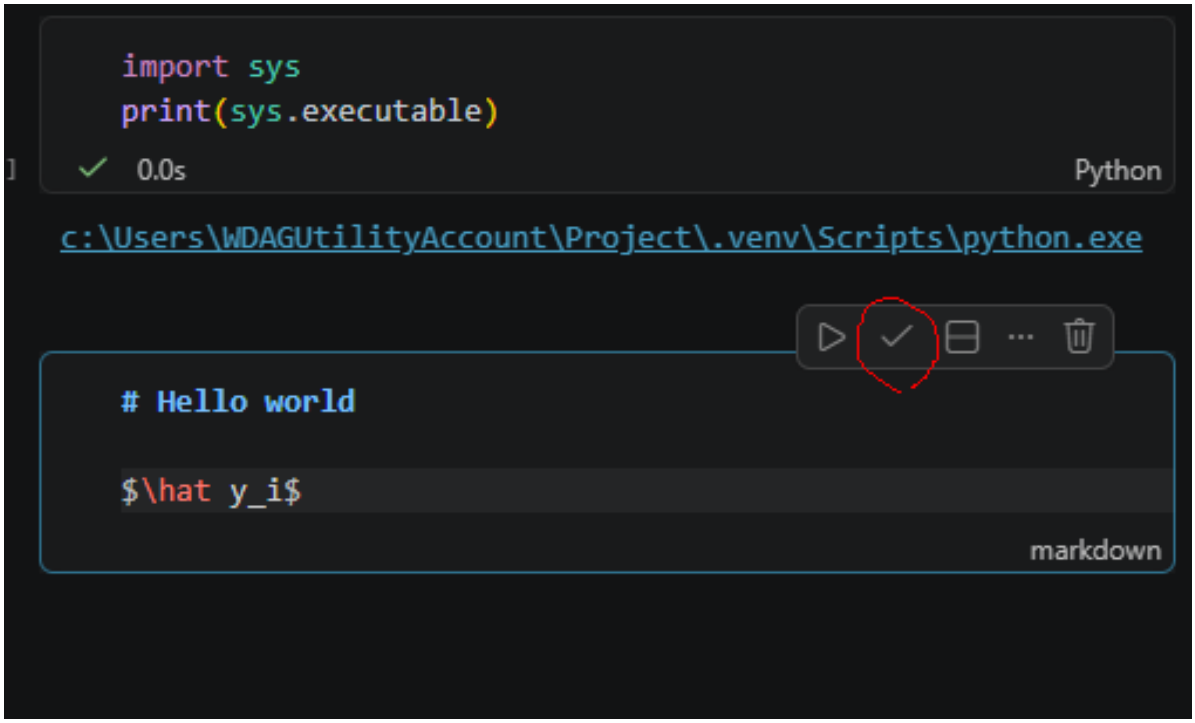
There are two types of cells in a notebook: Code cells and Markdown cells.

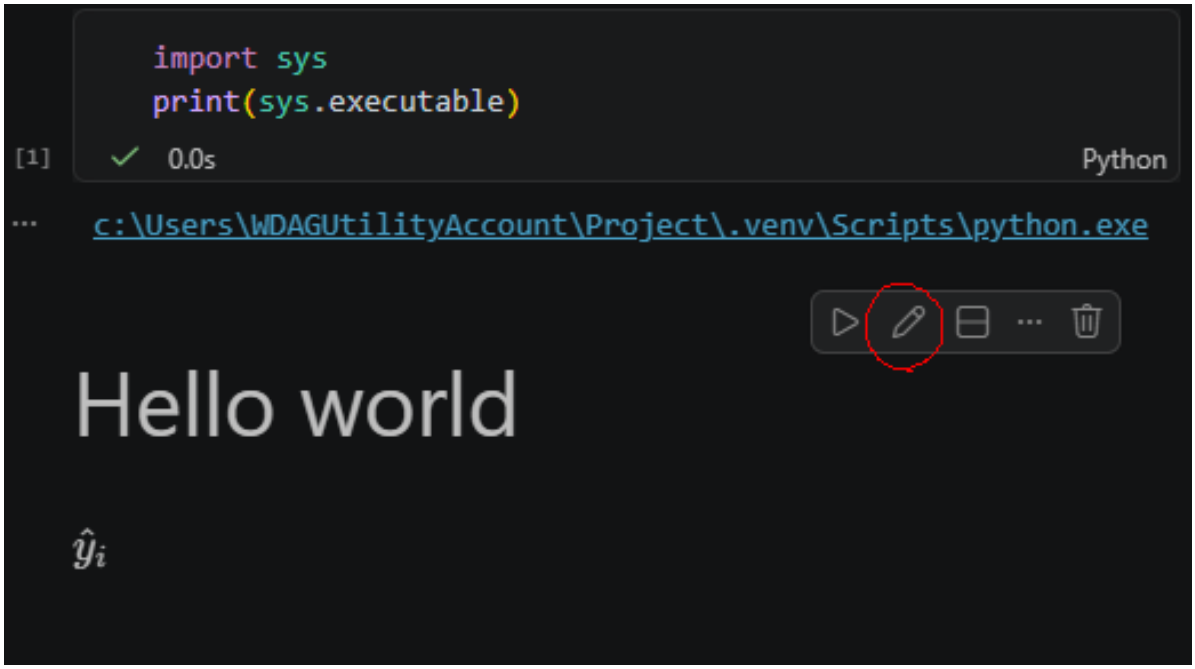
- Code cells are used to run Python code.
- Markdown cells are used to write text, explanations, equations, and documentation.

In statistical learning, code cells are typically used for data analysis, model fitting, and computations, while Markdown cells are used to explain the methods, interpret the results, and document the workflow.

A notebook combines code, output, and narrative in a single document, making it an ideal environment for data science and machine learning.

> **Tip**
>
> You can always click the small `Edit` / `Stop Editing` button to preview how your Markdown text will be rendered.





Markdown is a lightweight text format. In this course, we will use only the most basic Markdown syntax.

B.1 Text Formatting

Markdown Syntax	Output
`*italics*, **bold**, ***bold italics***`	*italics*, **bold**, ***bold italics***
`superscript^2^ / subscript~2~`	superscript^2 / subscript$_2$
`~~strikethrough~~`	~~strikethrough~~
`` `verbatim code` ``	`verbatim code`

B.2 Headings

Markdown Syntax	Output
`# Heading 1`	**Heading 1**

Markdown Syntax	Output
`## Heading 2`	**Heading 2**
`### Heading 3`	**Heading 3**

> **Tip**
>
> Note that for heading, there is a space after `#`.

B.3 Lists

Markdown Syntax	Output

`* unordered list`
`  + sub-item 1`
`  + sub-item 2`
`    - sub-sub-item 1`
	• unordered list
	  – sub-item 1
	  – sub-item 2
	  ∗ sub-sub-item 1

`1. ordered list`
`2. item 2`
`   i) sub-item 1`
`      A.  sub-sub-item 1`
	1. ordered list
	2. item 2
	  i) sub-item 1
	  A. sub-sub-item 1

> **Tip**
>
> Note that for lists, there is a space after `.`.

B.4 Essential LaTeX

Use single dollar signs for inline math:

```
The fitted value is $\hat y_i$.
```

The fitted value is $\hat{y}_i$.

Use double dollar signs for displayed equations:

```
$$
Y = \beta_0 + \beta_1 X + \epsilon.
$$
```

$$Y = \beta_0 + \beta_1 X + \epsilon.$$

Common symbols:

```
$\hat y$        predicted value
$\bar x$        sample mean
$\beta_0$       coefficient
$X^\top X$      transpose
$\sum_{i=1}^n$  summation
```

- $\hat{y}$: predicted value
- $\bar{x}$: sample mean
- β_0: coefficient
- $X^\top X$: transpose
- $\sum_{i=1}^{n}$: summation

LaTeX supports a wide range of mathematical notation, but the basic symbols introduced here will be enough for all of the work in this course.

C Python Basics for `sklearn`

This note introduces only the Python syntax needed for the rest of these notes. The goal is not to cover Python as a full programming language. The goal is to make code using `numpy`, `matplotlib`, and `sklearn` readable.

C.1 Running Python Code

A Python code chunk looks like this:

```
# This is a comment

x = 10      # assignment uses =

a, b = 2, 3
a + b       # last expression is automatically displayed
```

```
5
```

> **Notebook**
>
> 1. When a code cell is executed, the value of the last expression is automatically displayed. Any explicit output commands, such as `print()`, will also produce output.
> 2. Use `#` comments only for short notes that help explain the code. Detailed explanations and discussion should be written in Markdown cells, where they are easier to read and maintain.

C.1.1 Indentation

Python uses indentation to mark code blocks. R uses braces `{}`; Python uses indentation.

For example, a `for` loop:

```
for k in [1, 3, 5]:
    print(k)
```

```
1
3
5
```

The indented block belongs to the loop. These notes use loops only when they are helpful for plotting or comparing models.

C.1.2 `if` and `for`

The logic of `if` statements and `for` loops is straightforward. When using them, pay attention to indentation. A colon `:` indicates the beginning of an indented code block.

```
x = 1
if x == 2:
    print('good')
else:
    print('not good')
```

```
not good
```

```
for k in range(4):
    print(k)
```

```
0
1
2
3
```

C.1.3 Functions and Objects

This is not an object-oriented programming course, so don't worry too much about the details. The important idea is the distinction between a function, a class, and an object created from a class.

- Function: performs an action and returns a result. Called with parentheses: `round(3.14, 2)`

- Class: a blueprint for creating objects.
- Object: stores information and provides methods. Accessed with a dot: `object.method()`

Example in scikit-learn:

```
model = LinearRegression()
```

Here

- `LinearRegression` is the class;
- `LinearRegression()` creates a linear regression model object;
- the object is assigned to the variable `model`.

```
model.fit(X_train, y_train)
coef = model.coef_
```

Here

- `fit()` is method;
- `coef_` stores information about the fitted model;
- methods are called using `object.method(...)`;
- stored information is accessed using `object.attribute`.

In almost all `scikit-learn` code, you should work with objects rather than classes. A common mistake is to pass a class where an object is required.

```
pipe = make_pipeline(StandardScaler, LinearRegression)     # wrong
pipe = make_pipeline(StandardScaler(), LinearRegression()) # correct
```

The same idea applies to most `scikit-learn` components, including transformers, models, and pipelines. In general, if something is used as a step in a pipeline, you should create an object by adding `()`.

🔥 `.` (Dot)

The meaning of `.` is different in Python and R.

- In Python, `.` is used to access methods and attributes of an object.

```
model.fit(X, y)
model.coef_
```

- In R, `.` is usually just part of a variable or function name.

```
my.variable <- 1
```

Do not interpret dots in R names as object access.

C.1.4 Importing Packages

Python packages are loaded with `import`.

The usual imports in these notes are:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

The names on the right are aliases:

- `numpy` is used as `np`;
- `pandas` is used as `pd`;
- `matplotlib.pyplot` is used as `plt`.

After importing a package this way, you access its functions using the package name, e.g.

- `np.array([1, 2, 3])`
- `pd.DataFrame({"x": [1, 2, 3]})`
- `plt.plot([1, 2, 3], [4, 5, 6])`

> **Note**
>
> When a package is imported with `import ... as ...`, you must use the alias to access its functions and classes instead of the original name.

Sometimes only specific functions or classes are imported.

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
```

These objects can then be used directly.

```
model = LinearRegression()
```

> **Note**
>
> After `from sklearn.linear_model import LinearRegression`, you may use `LinearRegression()` directly, but other objects from `sklearn.linear_model` are not automatically imported.

C.2 Data structure

C.2.1 Basic Objects

Python has several basic data types.

```
x = 3          # integer
y = 2.5        # float
name = "tree"  # string
flag = True    # Boolean
```

Use `type()` to check the type of an object.

```
type(y)
```

```
float
```

Python is case-sensitive. `x` and `X` are different objects.

C.2.2 Lists

A list is an **ordered collection** of objects. Lists are commonly used to store variable names, column names, or collections of parameter values.

Purpose	Example	Explanation
Create a list	`features = ["age", "income", "education"]`	Creates a list containing three strings.
Access the first element	`features[0]`	Returns `"age"`. Python indexing starts at `0`.
Access the last element	`features[-1]`	Returns `"education"`. Negative indices count from the end.

Purpose	Example	Explanation
Select several elements	`features[0:2]`	Returns `["age", "income"]`. The stop index is not included.
Add an element	`features.append("gender")`	Adds `"gender"` to the end of the list.
Count elements	`len(features)`	Returns the number of elements in the list.

> **Note**
>
> Unlike R, Python uses zero-based indexing, so the first element has index `0`.

C.2.3 Dictionaries

A dictionary stores **key-value pairs**. It is similar to a named list in R and is commonly used to store settings, options, and model parameters.

Purpose	Example	Explanation
Create a dictionary	`params = {"n_neighbors": 5, "weights": "uniform"}`	Creates a dictionary with two keys and their values.
Access a value	`params["n_neighbors"]`	Returns `5`.
Add or update a value	`params["metric"] = "euclidean"`	Adds a new key-value pair or updates an existing one.
Count entries	`len(params)`	Returns the number of key-value pairs.
Store model settings	`{"max_depth": 3, "min_samples_leaf": 5}`	A common use in machine learning.
Store parameter grids	`{"max_depth": [2,3,4]}`	Often used when tuning hyperparameters.

A typical parameter grid in `scikit-learn` looks like:

```
param_grid = {
    "max_depth": [2, 3, 4],
    "min_samples_leaf": [1, 5, 10],
}
```

For pipelines, parameter names often use two underscores:

```
param_grid = {
    "model__C": [0.1, 1, 10],
}
```

> **Note**
>
> In a pipeline, a parameter name of the form `step__parameter` refers to a parameter inside a specific pipeline step. For example, `model__C` means the parameter `C` of the pipeline step named `"model"`. More terminologies will be discussed in related sections.

C.3 `numpy.array`

The most important data structure for numerical computation in Python is `numpy.array`.

```
import numpy as np

x = np.array([1, 2, 3, 4])
x
```

```
array([1, 2, 3, 4])
```

Arrays have a shape and a dimension.

```
print(x.shape)
print(x.ndim)
```

```
(4,)
1
```

So `x` is a 1D array containing 4 elements.

C.3.1 2D Arrays

A 2D array is similar to a matrix.

```
X = np.array([
    [1, 2],
    [3, 4],
    [5, 6]
])

X
```

```
array([[1, 2],
       [3, 4],
       [5, 6]])
```

```
print(X.shape)
print(X.ndim)
```

```
(3, 2)
2
```

Here `X` is a 2D array with 3 rows and 2 columns.

C.3.2 Indexing

Use `[row, column]` indexing.

```
X[0, 1]      # row 0, column 1
X[:, 0]      # first column
X[1, :]      # second row
```

The symbol `:` means "all indices" along that axis.

Slicing can select ranges.

```
X[0:2, 1]      # first 2 rows, column 1
X[0, 0:2]      # row 0, first 2 columns
X[0:2, 0:2]    # upper-left 2x2 block
```

> **Note**
>
> `X[:, 0]` returns a 1D array with shape `(n,)`, not a column vector.
> To obtain a column vector, use
>
> ```
> X[:, [0]]
> # or
> X[:, 0].reshape(-1, 1)
> ```
>
> which both have shape `(n, 1)`.

C.3.3 Common Operations

Arrays support vectorized arithmetic.

```
x + 1
x * 2
np.mean(x)
```

Operations are applied elementwise.

C.3.4 Axis-wise

Many functions accept an `axis=` argument.

```
X              # show X
```

```
array([[1, 2],
       [3, 4],
       [5, 6]])
```

Computes the sum across rows:

```
X.sum(axis=0)
```

```
array([ 9, 12])
```

Computes the mean across columns:

```
X.mean(axis=1)
```

```
array([1.5, 3.5, 5.5])
```

A useful rule:

- `axis=0`: operate down rows (one result per column)
- `axis=1`: operate across columns (one result per row)

C.3.5 Reshaping

A 1D array can be converted into a matrix. Consider a 1D array

```
x = np.array([1, 2, 3, 4])
```

To create a column vector:

```
x.reshape(-1, 1)
```

```
array([[1],
       [2],
       [3],
       [4]])
```

To create a row vector:

```
x.reshape(1, -1)
```

```
array([[1, 2, 3, 4]])
```

The value `-1` means "infer this dimension automatically."

A common pattern is

```
test_x = np.linspace(0, 1, 101).reshape(-1, 1)
```

On the other side, to flatten an array back to 1D:

```
X.reshape(-1)
```

```
array([1, 2, 3, 4, 5, 6])
```

`NumPy` reads entries row by row and produces a one-dimensional array.

> **Tip**
>
> Other flattening methods include `.ravel()` and `.flatten()`. The difference is about how copy-and-view is handled. We mainly use `.reshape(-1)` because it makes the resulting shape explicit.

> **Note**
>
> One common source of bugs in statistical learning is using arrays with incorrect shapes. In `scikit-learn`, predictors `X` are usually stored as a 2D array and responses `y` as a 1D array. Other libraries may use different conventions, and the required shapes often depend on the specific model or loss function.
> When shape-related errors occur, always check the documentation of the library you are using.

C.3.6 Arrays versus Lists

Lists are general-purpose containers, and its addition concatenates. For example

```
[1, 2, 3] + [4, 5, 6]
```

```
[1, 2, 3, 4, 5, 6]
```

Arrays are designed for numerical computation. Its operations are align with the regular math operations. Therefore for numerical work and machine learning, use `numpy.array` whenever possible.

C.4 Random Numbers

These notes often use `numpy` random number generators for simulation.

```
rng = np.random.default_rng(1)

x = rng.normal(size=5)
x
```

```
array([ 0.34558419,  0.82161814,  0.33043708, -1.30315723,  0.90535587])
```

> **Note**
>
> The number `1` is a seed. It makes the random output reproducible.
> On a single machine, if the seed is fixed the results are supposed to be the same.
> You may choose any number as the seed. You may use the same seed when you want to see the same result.

Some common random functions are:

```
rng.normal(loc=0, scale=1, size=5)
```

```
array([ 0.44637457, -0.53695324,  0.5811181 ,  0.3645724 ,  0.2941325 ])
```

```
rng.uniform(0, 1, size=5)
```

```
array([0.75351311, 0.53814331, 0.32973172, 0.7884287 , 0.30319483])
```

For a two-dimensional predictor matrix:

```
X = rng.normal(size=(10, 3))
X.shape
```

```
(10, 3)
```

C.5 `pandas`

`pandas` can be treated as a wrapper of `numpy.array` which make it more readable. The complete `pandas` is very complicated. Here we only mention a few very important concepts.

To import `pandas`, in most cases we set `pd` as its alias.

```
import pandas as pd
```

C.5.1 `DataFrame` and `Series`

In `pandas`, `DataFrame` is the 2D data table, and `Series` is the 1D column. There are many ways to create a `DataFrame`. Here we focus on one of them, that is to convert a `dict` into a `DataFrame`.

```
results = {
    'round': [1, 2, 3, 4, 5],
    'score': [0.22, 0.33, 0.44, 0.55, 0.66],
    'mse': [0.12, 0.11, 0.10, 0.09, 0.08]
}
df_res = pd.DataFrame(results)
df_res
```

	round	score	mse
0	1	0.22	0.12
1	2	0.33	0.11
2	3	0.44	0.10
3	4	0.55	0.09
4	5	0.66	0.08

A single column is a `Series`.

```
df_res['score']
```

```
0    0.22
1    0.33
2    0.44
3    0.55
4    0.66
Name: score, dtype: float64
```

Note that similar to the `numpy.array` case, if we select the column as a set (even if the set contains only 1 element), we will get a 2D object, which is a `DataFrame` in this case.

```
df_res[['score']]
```

	score
0	0.22
1	0.33
2	0.44
3	0.55
4	0.66

C.5.2 Read and write files

In `scikit-learn`, predictors and responses can be stored either as `numpy` arrays or as `pandas` objects. Throughout this course, many examples use `numpy`, but most code works equally well with `pandas` data structures.

When working with external datasets, a common workflow is:

1. Read the data into a `DataFrame`.
2. Perform data cleaning and preprocessing.
3. Extract predictors and responses.
4. Fit a statistical learning model.

Two of the most common tabular file formats are CSV files and Excel spreadsheets.

> **CSV**
>
> CSV stands for comma-separated values. A CSV file is a plain-text file that stores tabular data using a separator between columns.
> Common separators include:
>
> - `,` (comma)
> - `;` (semicolon)
> - `\t` (tab)
>
> Consider the following file `example.csv`.
>
> ```
> round   score   mse
> 1   0.220000    0.120000
> 2   0.330000    0.110000
> 3   0.440000    0.100000
> 4   0.550000    0.090000
> 5   0.660000    0.080000
> ```

In Python we could use `pd.read_csv()` to read it.

```
import pandas as pd
df = pd.read_csv('example.csv')
df
```

	round\tscore\tmse
0	1\t0.220000\t0.120000
1	2\t0.330000\t0.110000
2	3\t0.440000\t0.100000
3	4\t0.550000\t0.090000
4	5\t0.660000\t0.080000

The result is incorrect because this file uses tab characters (`\t`) rather than commas as separators.
Specify the separator explicitly:

```
df = pd.read_csv('example.csv', sep='\t')
df
```

	round	score	mse
0	1	0.22	0.12
1	2	0.33	0.11
2	3	0.44	0.10
3	4	0.55	0.09
4	5	0.66	0.08

Excel

Excel spreadsheets can be read using `pd.read_excel()`. We provide the exmaple.xlsx as an example.

```
import pandas as pd
df = pd.read_excel('example.xlsx')
df
```

	round	score	mse
0	1	0.22	0.12

1	2	0.33	0.11
2	3	0.44	0.10
3	4	0.55	0.09
4	5	0.66	0.08

Unlike CSV files, Excel files are stored in a binary format. Therefore, `pandas` requires an additional package called an engine to read and write them. The most commonly used engine is `openpyxl`.

```
uv add openpyxl
```

In this course, `openpyxl` is already included in the project's `pyproject.toml`, so no additional installation is required.

Writing files is similar to reading files. The main difference is that writing starts from an existing `DataFrame` instead of starting from `pd`.

```
df.to_csv('example2.csv')
df.to_excel('example2.xlsx')
```

C.5.3 Relative paths

In this course, always use relative paths when reading or writing files.

A relative path starts from the current working directory rather than from the root of the file system.

Suppose your working directory is

```
C:/Users/WDAGUtilityAccount/Project/
```

and you have the following structure:

```
Project/
   example.csv
   foldername/
```

Then you can read and write files using:

```
df = pd.read_csv("example.csv")
df.to_csv("foldername/result.csv")
```

Notice that neither path begins with a drive name such as `C:` or `D:`.

If your files are stored outside the working directory, **copy them into the working directory** before reading them.

> **Note**
>
> Although absolute paths such as
>
> ```
> pd.read_csv("D:/Files/example.csv")
> ```
>
> often work on your own computer, they may fail on another computer because the file locations are different.
> Using relative paths makes your code more portable and easier to share.

C.6 `matplotlib` Basics

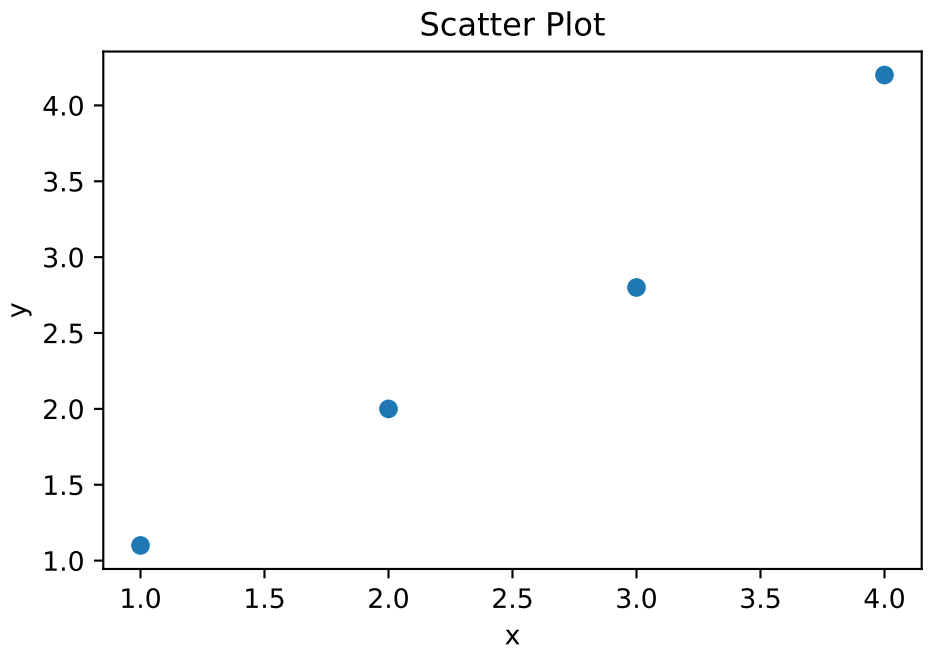
These notes use only basic `matplotlib`. The standard import is:

```
import matplotlib.pyplot as plt
```

We provide some code examples here. Most of the commands are straightforward. Please check the official document if you need more details.

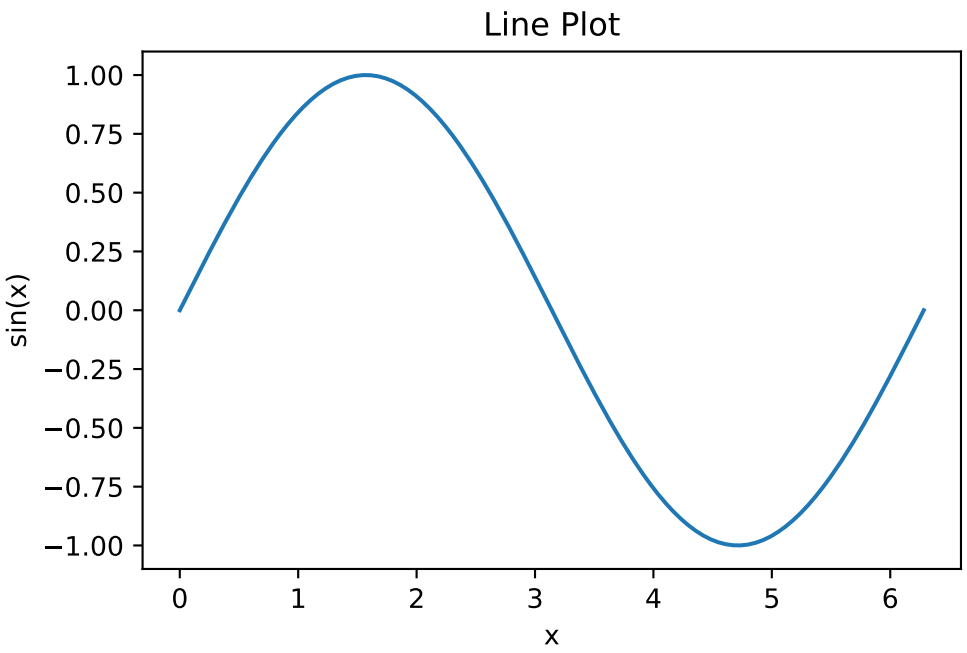
C.6.1 Scatter Plot

```
x = np.array([1, 2, 3, 4])
y = np.array([1.1, 2.0, 2.8, 4.2])

plt.scatter(x, y)
plt.xlabel("x")
plt.ylabel("y")
plt.title("Scatter Plot");
```



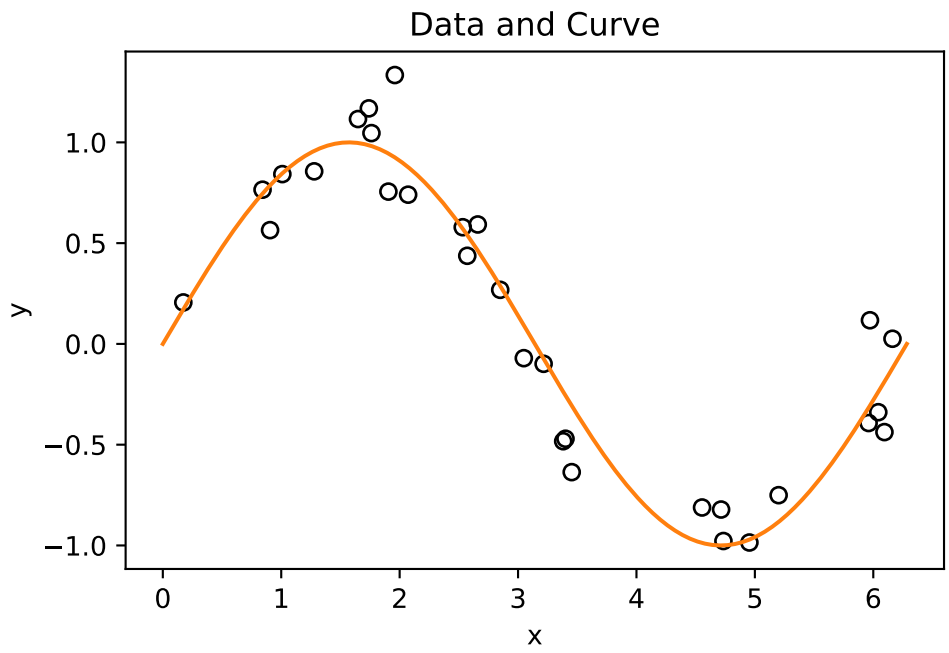
C.6.2 Line Plot

```
x_grid = np.linspace(0, 2 * np.pi, 200)
y_grid = np.sin(x_grid)

plt.plot(x_grid, y_grid)
plt.xlabel("x")
plt.ylabel("sin(x)")
plt.title("Line Plot");
```



C.6.3 Scatter and Line Together

```
rng = np.random.default_rng(1)

x = rng.uniform(0, 2 * np.pi, size=30)
y = np.sin(x) + rng.normal(scale=0.2, size=30)
x_grid = np.linspace(0, 2 * np.pi, 200)

plt.scatter(x, y, facecolors="none", edgecolors="black")
plt.plot(x_grid, np.sin(x_grid), color="C1")
plt.xlabel("x")
plt.ylabel("y")
plt.title("Data and Curve");
```

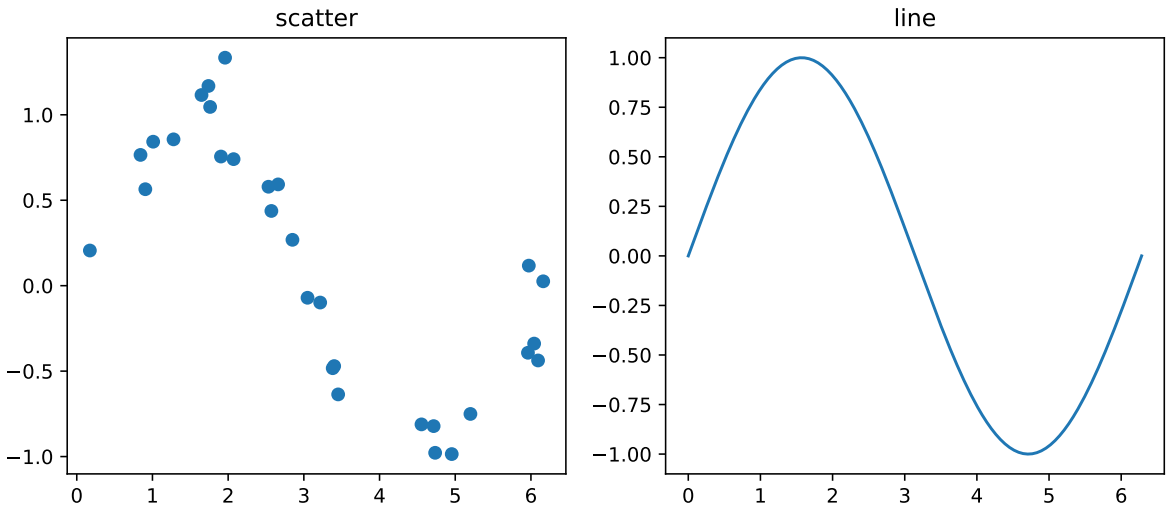


This pattern appears often when we plot data points and a fitted curve.

C.6.4 Multiple Plots

Use `plt.subplots` for multiple panels.

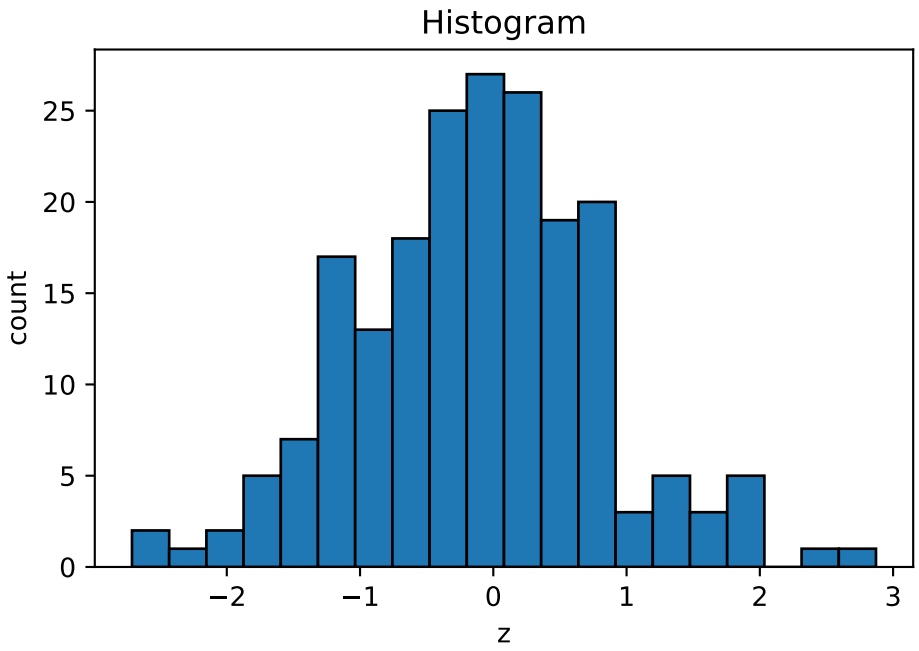
```
fig, axs = plt.subplots(1, 2, figsize=(10, 4))

axs[0].scatter(x, y)
axs[0].set_title("scatter")

axs[1].plot(x_grid, np.sin(x_grid))
axs[1].set_title("line");
```



> **Note**
>
> Notice that when using an axes object, many plotting commands have different names. For example, `plt.title()` becomes `ax.set_title()`. Therefore, you cannot always replace `plt` with `ax` directly. For details, consult the official documentation.

C.6.5 Histograms

```
z = rng.normal(size=200)

plt.hist(z, bins=20, edgecolor="black")
plt.xlabel("z")
plt.ylabel("count")
plt.title("Histogram");
```



C.7 Common Errors

C.7.1 One-Dimensional `X`

The most common error is passing a one-dimensional predictor array to `sklearn`.

Bad	Good
`x = np.array([1, 2, 3, 4])model.fit(x, y)`	`X = x.reshape(-1, 1)model.fit(X, y)`

C.7.2 Class Name versus Model Object

Another common mistake is confusing an `sklearn` class with an object created from that class.

For example, `DecisionTreeClassifier` is the class itself.

Bad	Good
`model = DecisionTreeClassifiermodel.fit(X_train, y_train)`	`model = DecisionTreeClassifier()model.fit(X_train, y_train)`

C.7.3 Mixing Lists and Arrays

Use arrays for elementwise numerical operations.

```
[1, 2, 3] * 2
```

```
[1, 2, 3, 1, 2, 3]
```

Plain lists do not always behave like numerical vectors.

D Some Python concepts

D.1 Language

Python is a programming language. In other words, it is a collection of syntaxes.

D.2 Interpreters

Python needs to be interpreted into codes that computers can understand. Therefore there should be some programs that translate Python scripts. These programs are called *interpreters*.

`CPython` is the reference implementation of Python and the implementation most people use. It is written mainly in C and Python. New Python language features are normally developed and tested first in CPython.

Other implementations include `PyPy`, `Jython` and `IronPython`. They are useful in specific contexts, but they do not always support the same Python versions or the same package ecosystem as CPython. For example, PyPy focuses on speed through a JIT compiler, Jython integrates with the JVM but its current stable release is Python 2.7-based, and IronPython integrates with .NET but lags behind modern CPython versions.

Since `CPython` is the most widely used and best-supported implementation, it is the best choice, at least for beginners. And actually, if you have no idea about this topic, but you use Python, it is highly possible that you are using `CPython`.

> **Note**
>
> We mentioned "interpreter" here. There are mainly two types of implimentations of programming langauges: interpreters and compilers. There are also some additional types like just-in-time compilers which can be treated as combinations of the two.
> Python is often described as an *interpreted* language, but `CPython` first compiles source code to bytecode and then executes that bytecode in the Python virtual machine. For beginners, the practical point is that Python supports an interactive workflow and does not require a separate manual compile step.

D.3 REPL

There are two ways to use Python interpreter. One way is to execute a Python script stored in a file. The second way is called *the intereactive shell*, that Python interpreter read the input from user directly, and print the result immediately. The model is like code example: prompt the user for some code, and when they've entered it, execute it in the same process. This model is often called a REPL, or *Read-Eval-Print-Loop*.

Shell, *terminal*, *console* have different meanings in their original contexts. However, nowadays, especially when talking about Python intereactive shell, these terminologies are used interchangeably. They are referred to the frontend of the system. In other words, the main task for the Python intereactive shell is to handle the user inputs and communicate with the backend, which is also called a *kernel*. We won't distinguish the real differences between these terminologies. The kernel will be discussed in the next section.

The standard Python REPL can usually be started with `python` or `python3`. On Windows, `py` may also be available. To quit, use `quit()`, `exit()`, `Ctrl+D` on macOS/Linux, or `Ctrl+Z` then `Enter` on Windows.

> **Note**
>
> In the REPL model, the backend (evaluation) is basically handled by the Python interpreter. The frontend is dealing with the user interface. Some typically tasks include the *primary/secondary prompt* and multi-line commands. The original REPL is very limited.

D.4 `IPython`

`IPython` was initially designed as an Enhanced interactive Python shell. However after many year's development, the whole `IPython` project becomes too big to maintain as one single project. Therefore it is now split into many smaller projects. The two most popular projects are `IPython` and `Jupyter`. This is called the Big Split.

The current `IPython` play two fundamental roles:

- Terminal `IPython` as the familiar REPL;
- The `IPython` kernel (which is defined below) that provides computation and communication with the frontend interfaces, like the notebook.

The core idea in the design of `IPython` is to abstract and extend the notion of a traditional REPL environment by decoupling the evaluation into its own process. We call this process a *kernel*: it receives execution instructions from clients and communicates the results back to them.

This decoupling allows us to have several clients connected to the same kernel, and even allows clients and kernels to live on different machines. This two-process model is now used by most of the `Jupyter` project.

You can launch the `IPython` shell on the command line with the `ipython` command (which similar to `python` case requires `PATH` configuration), and quit the shell with `exit`/`exit()`/`quit`/`quit()` commands.

The reference Python kernel provided by `IPython` is called `ipykernel`. With `ipykernel` you may create and maintain multiple kernels.

D.5 `Jupyter`

`Jupyter` is an ecosystem for interactive computing. It originated from the IPython Notebook project and later evolved into a language-independent platform supporting many programming languages. The name **Jupyter** is inspired by **Julia**, **Python**, and **R**, the three languages that were central to many early data science workflows.

Today, Jupyter includes several user interfaces, such as JupyterLab, Jupyter Notebook, Jupyter Console, and QtConsole. In this course, however, the important idea is not a particular interface, but the relationship among a notebook frontend, a kernel, and the notebook file stored on disk.

A Jupyter notebook is saved as a file with extension `.ipynb`. Internally, the file uses a structured `JSON` format that stores:

- code cells;
- Markdown cells;
- outputs;
- notebook metadata.

When you run code in a notebook, the frontend sends the code to a kernel, which executes the code and returns the results. In a Python notebook, a kernel can be viewed as a Python process running in the background. It executes code, stores variables, and communicates with the notebook interface.

Different programming languages can be supported by different kernels. For example:

- `ipykernel` for Python;
- `IRkernel` for R;
- `IJulia` for Julia.

Kernel

A kernel is associated with a particular Python environment, but it is not the environment itself. For example, multiple kernels may use the same Python environment while maintaining completely separate variables and memory. Conversely, multiple notebook windows may connect to the same kernel and therefore share the same variables and computational state.
For this course, you can think of the notebook as the user interface, the kernel as the Python process that runs your code, and the `.ipynb` file as the notebook document saved on disk.

D.5.1 Multi-kernels setup

This section is mainly following the official document.

To install one `IPython` kernel, you may install `ipykernel` in the environment. If you want to have multiple `IPython` kernels for different venvs, you will need to specify unique names for the kernelspecs.

1. Activate the environment you want.
2. Install the kernel in the environment.

```
uv add ipykernel
uv run python -m ipykernel install --user --name myenv --display-name "Python (myenv)"
```

`--user` means that the kernel is installed in the user's folder instead of a system folder, and it can be removed. The `--name` value (in this case it is `myenv`) is used by Jupyter internally. These commands will overwrite any existing kernel with the same name. `--display-name` is what you see in the notebook menus.

3. You could use the command to find all kernels installed in your system.

```
uv run jupyter kernelspec list
```

Available kernels are shown, as well as the path to the kernel configuration file `kernel.json`. The most important configuration is the path to the Python interpreter executatable file.

Notice that the path in `argv` points to the Python interpreter used by this kernel. Different kernels may point to different Python environments, or multiple kernels may point to the same environment.

Listing D.1 `kernel.json`

```
{
 "argv": [
  "C:\\Users\\WDAGUtilityAccount\\Project\\.venv\\Scripts\\python.exe",
  "-Xfrozen_modules=off",
  "-m",
  "ipykernel_launcher",
  "-f",
  "{connection_file}"
 ],
 "display_name": "(myenv)",
 "language": "python",
 "metadata": {
  "debugger": true
 },
 "kernel_protocol_version": "5.5"
}
```